Explainable Financial Alerts for Retail Investors: Integrating Statistical Anomaly Detection and News–Market Impact Correlation

Henrique José da Silva Santos

Student No.: 1180934

A dissertation submitted in partial fulfillment of the requirements for the degree of Master of Science, Specialisation Area of

Supervisor: Luís Gomes
Co-Supervisor: Rafael Silva

Evaluation Committee:
President:
[A definir]

Members:
[A definir]

Porto, July 22, 2026

Statement Of Integrity

I hereby declare that I have conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore, the work presented in this document is original and was authored by me, having not been previously used for any other purpose.

I declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

Use of Artificial Intelligence. In accordance with the P.PORTO/ISEP guidelines, I declare that artificial intelligence tools (notably Claude Code) were used during the development of this work to assist with the drafting and editing of the text and with software development. I directed the work and critically reviewed and validated all outputs; the ideas, decisions and final content are my own and are my responsibility.

ISEP, Porto, July 22, 2026

Dedicatory

Abstract

This dissertation presents InvestiGator, an explainable (XAI-first) financial-alert system for retail investors in the US equity market (NYSE and NASDAQ). The system raises Telegram alerts from two independent triggers (abrupt market movements, detected as statistical anomalies, and incoming financial news) and, for every alert, exposes a fully traceable explanation: the detected event, the reasoning applied, the sources, and historical precedents. Its core is a news–market correlation engine that, given a news item, retrieves analogous historical news from a knowledge base and measures the impact those precedents had on prices, event-study style and without lookahead. As an Artificial Intelligence Engineering contribution, the work integrates, applies and critically evaluates existing components (sentence-embedding retrieval, statistical anomaly detection, supervised triage and transparent explanation) in a functional, explainable and reproducible system. On real data, semantic retrieval clearly outperformed lexical and trivial baselines (cross-ticker precision@5 of 0.51 against 0.35 lexical and 0.24 random), and the volatility-normalised detector fired far more consistently across stocks than a fixed threshold. A materiality-triage model trained on 79,753 multi-year news–market examples nearly quadrupled within-budget alert precision, yet no text model beat the volatility-only baseline, a finding reported as it stands. Limitations and future work are stated explicitly.

Keywords: Explainable AI, Financial Alerts, Anomaly Detection, News Impact, Retail Investors, Financial NLP

Resumo

Esta dissertação apresenta o InvestiGator, um sistema de alertas financeiros explicável (XAI-first) para investidores de retalho no mercado acionista dos Estados Unidos (NYSE e NASDAQ). O sistema envia alertas via Telegram a partir de dois gatilhos independentes (movimentos abruptos de mercado, detetados como anomalias estatísticas, e novas notícias financeiras) e, para cada alerta, expõe uma explicação totalmente rastreável: o evento detetado, o raciocínio aplicado, as fontes e os precedentes históricos. O seu núcleo é um motor de correlação notícia–mercado que, dada uma notícia, recupera notícias históricas análogas de uma base de conhecimento e mede o impacto que esses precedentes tiveram nos preços, à maneira de um estudo de evento e sem lookahead. Enquanto contribuição de Engenharia de Inteligência Artificial, o trabalho integra, aplica e avalia criticamente componentes existentes (recuperação por embeddings de frases, deteção estatística de anomalias, triagem supervisionada e explicação transparente) num sistema funcional, explicável e reproduzível. Em dados reais, a recuperação semântica superou claramente as baselines lexical e triviais (precision@5 cross-ticker de 0,51, face a 0,35 de uma baseline lexical e 0,24 do acaso) e o detetor normalizado pela volatilidade disparou de forma muito mais consistente entre ações do que um limiar fixo. Um modelo de triagem de materialidade treinado em 79 753 exemplos notícia–mercado de vários anos quase quadruplicou a precisão dos alertas dentro do orçamento diário, mas nenhum modelo com texto bateu a baseline de só-volatilidade, resultado reportado tal como é. As limitações e o trabalho futuro ficam explícitos.

Palavras-chave: Explainable AI, Financial Alerts, Anomaly Detection, News Impact, Retail Investors, Financial NLP

Acknowledgement

Contents

List of Figures

List of Tables

List of Algorithms

List of Symbols

r_t	log-return of an asset on day t	—
μ_t	rolling mean of returns	—
σ_t	rolling standard deviation of returns	—
z_t	z-score of the return on day t	—
k	anomaly threshold (in standard deviations)	—
w	rolling-window length	d

List of Acronyms

AI	Artificial Intelligence.
NASDAQ	National Association of Securities Dealers Automated Quotations.
NYSE	New York Stock Exchange.
RQ	Research Question.

Chapter 1

Introduction

1.1 Contextualization

Most Americans now own stock, directly or through retirement accounts and mutual funds (Gallup 2025), and they invest in the world's largest equity market, worth US$62.2 trillion at the end of 2024 (Securities Industry and Financial Markets Association (SIFMA) 2025) (Figure 1.1). Trading happens mainly on the New York Stock Exchange (NYSE) and National Association of Securities Dealers Automated Quotations (NASDAQ), and a growing share of it now comes from ordinary people using commission-free mobile apps, not from professional institutions.

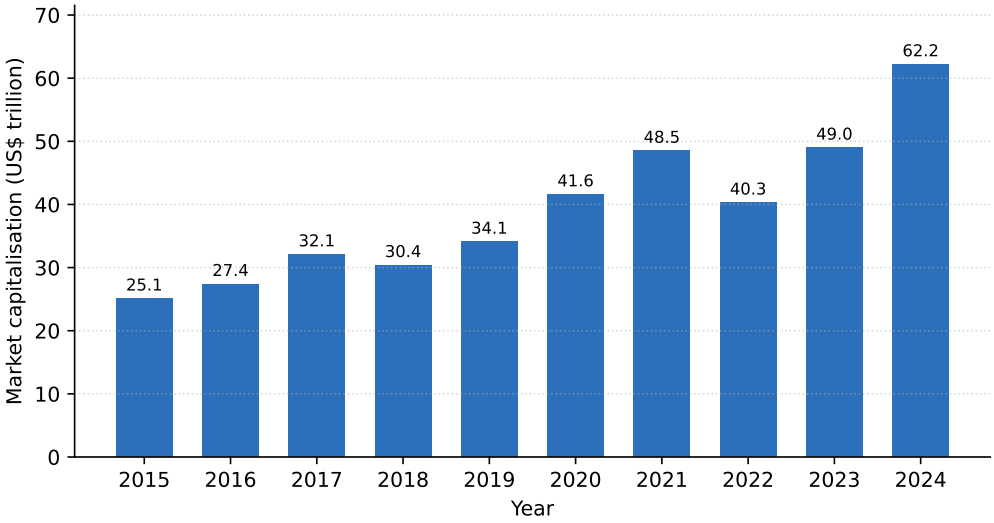

Figure 1.1: US equity market capitalisation, 2015–2024 (US$ trillion). Drawn from data reported in the SIFMA 2025 Capital Markets Fact Book (Securities Industry and Financial Markets Association (SIFMA) 2025).

These non-professional, or "retail", investors are the audience of this work, and their position is hard. Markets move continuously across thousands of securities, and relevant news arrives all through the trading day, yet retail investors have none of the tools that banks and funds take for granted. Ownership is uneven too: it reaches 87% of households earning above US$100,000 but only 28% below US$50,000 (Gallup 2025).

Meanwhile, artificial intelligence has spread quickly through finance: a 2026 survey found 81% of financial firms adopting Artificial Intelligence (AI) and 71% already using generative AI (Cambridge Centre for Alternative Finance 2026). But most of this technology is opaque,

rarely showing how it reaches a conclusion (Adadi and Berrada 2018; Barredo Arrieta et al. 2020). For someone who must act on an alert and live with the result, that opacity is the obstacle this dissertation removes: it lets the investor see *why* an alert was raised.

1.2 Problem Statement

A retail investor who wants to stay informed faces two everyday problems. The first is telling a genuinely unusual price move from ordinary volatility, which takes statistical judgement and constant attention. The second is reading what a news item might mean, which takes knowing how similar events played out before. Most people can do neither at scale, and the consumer apps that promise to help usually send bare alerts or hide their reasoning inside an opaque model.

This work takes a deliberately different stance: it does not predict prices, it *explains* events. For each abrupt move or relevant news item, the system shows what it detected, how, where the information came from, and which past events resemble the current one, together with what happened to prices afterwards (Figure 1.2).

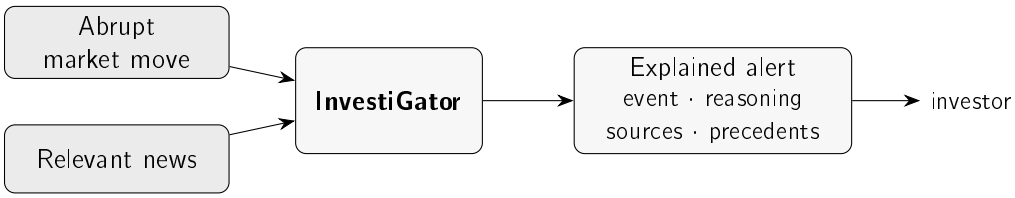

Figure 1.2: The two triggers, a sudden market move or a relevant news item, feed InvestiGator, which answers with an explained alert the investor can follow and check, not just a bare notification.

1.3 Research Questions and Objectives

The aim is to design, build and evaluate an explainable financial-alert system for US retail investors, driven by two independent triggers, a sudden market move and an incoming news item, and delivered over Telegram. Four research questions guide the work:

- **Research Question (RQ)1.** Can transparent statistical methods detect abrupt market movements consistently enough to be useful to a retail investor, while staying explainable?
- **RQ2.** Can a news–market correlation engine retrieve genuinely analogous historical news better than trivial baselines, and quantify the observed price impact of those precedents without lookahead?
- **RQ3.** Can the resulting explanations be made faithful to what the system actually computes, and useful to a retail investor?
- **RQ4.** Can a supervised model, trained on historical news and market context, usefully prioritise which news items deserve an alert, beyond simple volatility baselines, while never predicting price direction?

These questions map onto five concrete objectives:

- build an explainable alerting pipeline driven by the two triggers;

- evaluate the anomaly detector against transparent baselines;
- evaluate the retrieval of historical news against trivial baselines;
- assess whether the explanations are faithful and useful;
- train, calibrate and honestly evaluate a materiality-triage model that ranks news alerts.

The system is built incrementally, in its simplest defensible form first, a methodological choice explained in Chapter 3.

1.4 Scientific Contributions

This is a dissertation in Artificial Intelligence *Engineering*: the contribution is not a new algorithm but the disciplined assembly of existing ones into a system that works, explains itself, and can be reproduced. Four contributions stand out:

- A reproducible method for relating news to market impact, pairing sentence-embedding retrieval with event-study windows, with the similarity measure and the time horizons stated explicitly and no information about the future allowed to leak in.
- **InvestiGator**, an alerting system that shows its whole reasoning chain: the detected event, the supporting evidence, the sources, and the relevant historical precedents. Because each explanation is built directly from the quantities the system computes, the text cannot drift from the computation.
- A trained and calibrated *materiality-triage* model: supervised learning over historical news and market context that estimates how often news with a given profile was followed by an abnormal move, labelled by the system's own event-study code, evaluated against a volatility baseline, and integrated into the alerts as ranked, explained evidence rather than a forecast.
- A critical, honest evaluation of each core component on real data, against simple and lexical baselines, with the results, the ablations and the limitations all reported plainly.

1.5 Document Structure

The rest of the dissertation is written to be read in order, each chapter answering one question:

- **Chapter 2 (State of the Art):** what is already known, and what does this work choose?
- **Chapter 3 (Methods and Materials):** how is the system built, and how is it judged?
- **Chapter 4 (InvestiGator):** what is the system, and how do its data, parts, flow and decisions fit together?
- **Chapter 5 (Case Studies):** does it work on real data?
- **Chapter 6 (Conclusions):** what was learned, and what comes next?

Chapter 2

State of the Art

This chapter has one job: to justify every design choice in InvestiGator against the alternatives. Each section asks what a field offers and ends with what InvestiGator takes from it. The order is: how retail investors behave; anomaly detection; explainable AI; trust in automation; financial text representation; information retrieval; event studies; and case-based reasoning. The recurring finding is simple. The building blocks are mature, but an integrated, explainable system for the retail investor is still rare.

2.1 Retail Investing and Information Overload

How do retail investors behave, and what should an alert give them?

Behavioural finance shows that real investors depart systematically from the rational ideal (Barberis and Thaler 2003). One root cause is *loss aversion*: we feel losses more than equal gains (Kahneman and Tversky 1979), which makes investors especially sensitive to bad news, and a timely, well-explained alert valuable.

The strongest pattern is *attention*. Barber and Odean (2008) show that individuals are net buyers of attention-grabbing stocks (those in the news, with unusual volume, or extreme one-day returns): facing thousands of choices, they fall back on whatever caught their eye. Attention can even be measured through search volume (Da, Engelberg, and Gao 2011). This is decisive for the design, because a sharp price move and a salient headline are exactly InvestiGator's two triggers, so an alert that adds context lands where attention-driven decisions are made.

News also moves markets, and its text can be measured. Tetlock (2007) builds a daily pessimism index from a newspaper column and links it to price pressure and trading volume, an early proof that textual signals relate to market outcomes, which is the premise of InvestiGator's news engine. This audience is large and active too: even through the March 2020 crash, retail investors on one commission-free platform added to their holdings rather than panicking (Welch 2022), consistent with the broad ownership and market scale reported in Chapter 1 (Gallup 2025; Securities Industry and Financial Markets Association (SIFMA) 2025).

For InvestiGator: retail investors are attention-constrained and news-driven, so the system does not predict the market; it lowers the cost of *interpreting* the events that already grab attention, with a transparent explanation and real historical evidence.

2.2 Anomaly Detection in Financial Markets

How do you tell an abnormal price move from ordinary noise?

InvestiGator's first trigger is an anomaly detector, so it sits in the mature field of anomaly detection. The standard taxonomy (Chandola, Banerjee, and Kumar 2009) separates *point*, *contextual* and *collective* anomalies, and groups methods into statistical, distance/density, and machine-learning families (Figure 2.1). A sharp single-day return is a contextual point anomaly: abnormal relative to the asset's own recent behaviour.

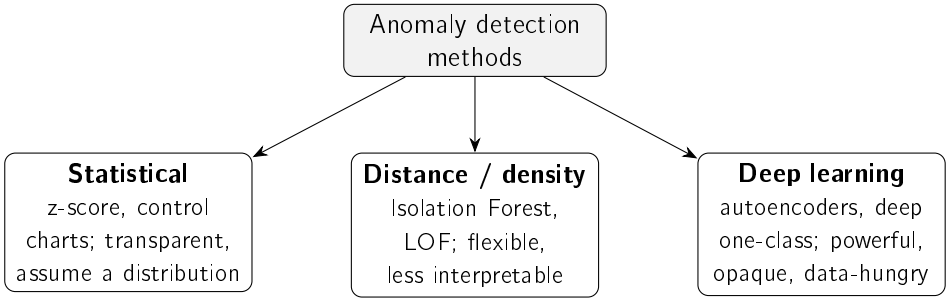

Figure 2.1: Families of anomaly-detection methods relevant to this work, following Chandola, Banerjee, and Kumar (2009) and Pang et al. (2021). Interpretability decreases, and modelling capacity increases, from left to right.

The simplest detector is a rolling z-score: flag a return that deviates far from its recent mean and standard deviation. It is attractive because it is transparent and easy to justify. Its one assumption, a locally stable distribution, is handled by the rolling window, which tracks *volatility clustering*, the well-known tendency of calm and turbulent periods to arrive in runs (Bollerslev 1986; Engle 1982). The rolling standard deviation is, in effect, a simpler non-parametric version of that idea: it adapts to the current regime without fitting a model the user could not scrutinise. Between the two sits the exponentially weighted moving average of squared returns (the RiskMetrics estimator), which weights the recent past more heavily than the distant past but still estimates no parameters per instrument. A full GARCH fit is declined here under defensible simplicity, but the claim is not left as opinion: the EWMA variant, the first step on the road to GARCH, is put through the same evaluation protocol in Chapter 5, so the trade-off is measured rather than asserted.

More expressive detectors trade interpretability for capacity. Isolation Forest isolates outliers by random partitioning and scales to high dimensions (F. T. Liu, Ting, and Zhou 2008); the Local Outlier Factor scores how isolated a point is within its neighbourhood (Breunig et al. 2000); deep detectors learn a model of normality (Pang et al. 2021). All are powerful but harder to read off, and their edge shows most in high-dimensional data, not a single return series. Rather than dismissing them on those grounds alone, Chapter 5 evaluates both Isolation Forest and the Local Outlier Factor under the same causal protocol as the statistical rule, so the comparison is empirical. In finance, detection is anyway dominated by *unsupervised* methods, because labelled anomalies are scarce (Ahmed, Mahmood, and Islam 2016), a constraint that also shapes how InvestiGator is evaluated (Chapter 5). Table 2.1 compares the families.

For InvestiGator: the signal is one low-dimensional return series that must be explained to a non-expert, so the transparent z-score wins; the more expressive families are noted as alternatives whose opacity the problem does not justify.

Table 2.1: Anomaly-detection approaches relevant to this work.

Approach	How it works	Strengths / limitations
Statistical z-score (rolling) (Chandola, Banerjee, and Kumar 2009)	Flags returns that deviate from a rolling mean and standard deviation	Transparent and easy to explain; assumes a locally stable distribution
Isolation Forest (F. T. Liu, Ting, and Zhou 2008)	Isolates points via random partitioning (ensemble)	Scalable, handles high dimensionality; score is not directly interpretable
Local Outlier Factor (Breunig et al. 2000)	Scores isolation within a local density neighbourhood	Captures local structure; sensitive to neighbourhood size, score hard to read
Deep detectors (Pang et al. 2021)	Learn a model of normality (e.g. autoencoders)	High capacity for complex data; opaque, data-hungry, hard to justify

2.3 Explainable Artificial Intelligence

What kind of explanation should the system give?

Explainable AI (XAI) exists because the strongest models rarely show their reasoning. Surveys map the field and its central split (Adadi and Berrada 2018; Barredo Arrieta et al. 2020): *transparent* models, interpretable in themselves, versus *post-hoc* methods that explain a trained black box, which can be organised by what they explain (Guidotti et al. 2018) (Figure 2.2). A useful caution: "interpretability" is not one thing (Lipton 2018), so a system should say which kind it offers. InvestiGator offers transparency of its decision logic and traceable evidence, not a claim of full interpretability.

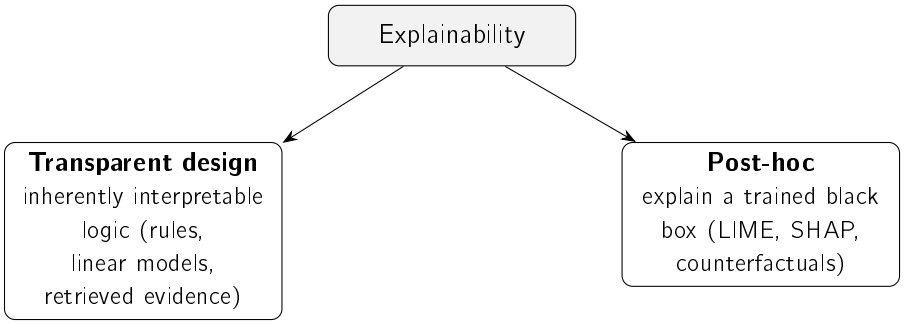

Figure 2.2: The two broad routes to explainability (Barredo Arrieta et al. 2020; Guidotti et al. 2018): building models that are transparent by design, or explaining a black box after the fact. This work follows the former.

Two post-hoc techniques are widely used: LIME fits a simple local surrogate around one prediction (Ribeiro, Singh, and Guestrin 2016), and SHAP attributes a prediction to its features using Shapley values (Lundberg and S.-I. Lee 2017). Both are valuable when one is committed to a black box. But for high-stakes decisions, Rudin (2019) argues for dropping that commitment and using models that are interpretable in the first place, since a post-hoc explanation is itself an approximation that can mislead. InvestiGator follows this: it explains

primarily by transparent design (every quantity exposed, real precedents shown as evidence), with feature attribution as an optional extra.

Good explanations are also a human matter. People want *contrastive*, selective reasons, not an exhaustive dump (Miller 2019), which is why each alert shows a few relevant precedents and the exact quantities behind the decision. Explanations must also be evaluated, not assumed: Doshi-Velez and Kim (2017) make *fidelity*, how well an explanation reflects the real computation, a core criterion, and fidelity is the property InvestiGator can guarantee (Chapter 5). Table 2.2 summarises the options.

Table 2.2: Explainability approaches considered.

Method	**Type**	**Strengths / limitations**
LIME (Ribeiro, Singh, and Guestrin 2016)	Local surrogate, post-hoc	Intuitive, model-agnostic; explanations can be unstable, only locally faithful
SHAP (Lundberg and S.-I. Lee 2017)	Shapley-value attribution, post-hoc	Consistent, theoretically grounded; computationally costly
Transparent design (Barredo Arrieta et al. 2020; Rudin 2019)	Inherently interpretable logic	No approximation, faithful by construction; constrains model choice

For InvestiGator: pursue explainability by transparent design and judge it by fidelity; treat post-hoc attribution as a complement, not the foundation.

2.4 Trust and Appropriate Reliance in Decision Support

When should the investor actually trust an alert?

An explanation only helps if it produces *appropriate* reliance: not dismissing a sound warning, not acting blindly on a fallible one. The human-factors view (J. D. Lee and See 2004) calls the goal *calibrated* trust, reliance matched to the system's real competence, and names two failure modes, *disuse* (ignoring a good aid) and *misuse* (over-trusting a flawed one). With real money, both are costly.

Explanations are how trust gets calibrated, but they are not magic. Bansal et al. (2021) show that explanations can even cause *over-reliance*, with people accepting confident explanations of wrong answers. InvestiGator's response is concrete: explain with *verifiable evidence* (the actual z-score and its parameters; real precedents with their measured impact), and disclaim prediction on every alert.

For InvestiGator: transparency is not decoration but the mechanism for appropriate reliance, so the system shows checkable evidence rather than a persuasive story.

2.5 Financial NLP and Text Representation

How should the system represent the meaning of a headline?

To compare headlines, text must become numbers. The financial-NLP literature moves through three generations (Kearney and S. Liu 2014): lexicons, word embeddings, and contextual neural models.

Lexicons came first. Finance-specific word lists fix the fact that general sentiment dictionaries misread ordinary financial words such as "liability" or "tax" (Loughran and McDonald 2011); they are fully transparent but bounded by their vocabulary. Word embeddings then placed words in a vector space where proximity means similar use: word2vec from local context (Mikolov et al. 2013), GloVe from global co-occurrence (Pennington, Socher, and Manning 2014). Both give each word a single vector, blind to context.

The Transformer (Vaswani et al. 2017) removed that limit, and BERT (Devlin et al. 2019) produced deeply *contextual* representations. Comparing two texts with BERT directly is costly, though; Sentence-BERT (Reimers and Gurevych 2019) fixes this by giving each sentence one fixed-length vector comparable by a cheap cosine, exactly what precedent retrieval needs. At very large scale, exact search can be swapped for an approximate index (Johnson, Douze, and Jégou 2021) without changing the representation (future work). Two heavier options also exist: domain-tuned FinBERT models (Araci 2019; Y. Yang, Uy, and Huang 2020) and large financial LLMs such as BloombergGPT (Wu et al. 2023), both powerful but costlier, less transparent, and outside this work's free, reproducible constraint. Table 2.3 compares them.

Table 2.3: Text-representation approaches for financial news.

Approach	**Representation**	**Strengths / limitations**
Finance lexicons (Loughran and McDonald 2011)	Curated word lists / tone	Fully transparent; bounded by vocabulary
word2vec / GloVe (Mikolov et al. 2013; Pennington, Socher, and Manning 2014)	Static word vectors	Efficient; one vector per word, no context
BERT (Devlin et al. 2019; Vaswani et al. 2017)	Contextual token embeddings	Strong understanding; pairwise similarity is costly
Sentence-BERT (Reimers and Gurevych 2019)	Sentence embeddings (siamese)	Efficient cosine similarity; the choice in this work
FinBERT (Araci 2019; Y. Yang, Uy, and Huang 2020)	Domain-adapted language model	Finance-tuned; oriented to sentiment / communications
Financial LLMs (Wu et al. 2023)	Large generative model	Very capable; costly, opaque, outside free-tier scope

For InvestiGator: Sentence-BERT is the sweet spot, contextual meaning beyond lexicons and static vectors, yet cheap, comparable and free to run. One honesty note on the comparison: the evaluation in Chapter 5 measures Sentence-BERT against a *lexical* baseline, which brackets the question that matters (does meaning beat surface words?); word2vec and FinBERT variants are not run empirically, because static word vectors are strictly dominated

by contextual sentence embeddings for sentence comparison, and domain-tuned FinBERT is oriented to sentiment classification rather than sentence similarity, at a cost this work's free-and-reproducible constraint excludes. Where a claim is cheap to test, this work tests it; where it would buy little, the reason is stated instead.

2.6 Information Retrieval and Ranking Evaluation

Once headlines are vectors, how do you rank past news, and how do you tell if the ranking is good?

Ranking a corpus by relevance to a query is the defining problem of information retrieval (IR). The vector-space model (Salton, Wong, and C. S. Yang 1975) represents documents and queries as vectors ranked by cosine similarity; InvestiGator's embedding retrieval is the same idea with richer vectors. Classical *lexical* ranking weights informative terms, from TF-IDF to BM25 (Robertson and Zaragoza 2009), still a strong baseline. Its known weakness is *vocabulary mismatch*: a relevant document that uses different words scores as dissimilar. Semantic embeddings overcome exactly that, which is why InvestiGator compares against a lexical baseline (Chapter 5).

Because retrieval returns a ranked list, it is judged with rank-aware measures (Manning, Raghavan, and Schütze 2008); the most direct is *precision@k*, the fraction of the top k results that are relevant. InvestiGator adopts it because an alert can show only a handful of precedents, so what matters is whether the top few are relevant, reported against random, recency and lexical baselines.

For InvestiGator: retrieval reuses a mature representation-then-rank pipeline and the standard precision@k metric; the contribution is the application and the pairing with measured impact, not a new algorithm.

2.7 Event Studies and News–Market Impact

Having found analogous news, how do you measure what it did to the price, honestly?

The tool is the *event study*, introduced by Fama et al. (1969) and standardised for daily returns by Brown and Warner (1985), who set the practice of measuring returns over short post-event windows, the basis for InvestiGator's +1, +3 and +5-day horizons. The impact is measured strictly *after* the event, so it describes an outcome, not a forecast.

Why not just predict the price? Because public news is absorbed into prices almost at once, the efficient-market hypothesis (Fama 1970), so forecasting returns from public news is very hard by construction. InvestiGator therefore does not compete with market efficiency; it measures and explains the reactions that *already* followed comparable news, which is both more honest and more defensible for a non-expert.

Relating news *text* to impact is an active area: Tetlock (2007) is an early example, and a more recent strand tries to *predict* moves from news (Ding et al. 2015; Xing, Cambria, and Welsch 2018). InvestiGator deliberately stops short of prediction. Doing this at scale needs news aligned to prices, which is the contribution of the FNSPID dataset (Dong, Fan, and Peng 2024), used to build the knowledge base; its coverage and period limits are noted in the data card of Chapter 3.

For InvestiGator: pairing semantic retrieval with event-study impact over FNSPID is the methodological core, evidence about the past, never a forecast.

2.8 Case-Based Reasoning and Precedent Retrieval

What is the engine, in one familiar idea?

It is *case-based reasoning* (CBR). In the classic account (Aamodt and Plaza 1994), a CBR system *retrieves* similar past cases, *reuses* their outcomes, and optionally *revises* and *retains* them. InvestiGator is the retrieve-and-reuse core: each past headline plus its measured impact is a *case*; a new headline is the query; cosine similarity retrieves; the precedents' impact is reused as evidence. It stops before *revise*, so it never turns evidence into a prediction.

For InvestiGator: naming the engine as CBR shows it is a recognised paradigm, not ad hoc, and its "this resembles these past cases" explanations are naturally intelligible to a non-expert.

2.9 Learned Alert Triage: Supervised Materiality Classification

When dozens of headlines arrive in a day, which ones deserve an alert?

Retrieval answers "what does this resemble?", and the event study answers "what followed similar news?". Neither ranks the incoming stream. That is a *triage* problem, and it can be posed as supervised classification without breaking the no-forecast rule. The model estimates the probability that an *abnormal move of unusual size, in either direction*, follows a news item; the label is produced by the system's own event-study measurement, the market-adjusted return of Brown and Warner (1985) against an index proxy. The target is *materiality*, whether the market reacted at all. Direction and magnitude of the price path are never predicted, so the task stays on the defensible side of the efficient-market line drawn earlier in this chapter.

Two model families cover the interpretability–capacity trade-off. Logistic regression is the transparent choice: with standardised inputs, its log-odds are an exact additive sum of per-feature contributions. That makes it the kind of inherently interpretable model Rudin (2019) argues should be preferred when explanations matter. Gradient-boosted decision ensembles (Friedman 2001) are the stronger nonlinear learner and serve as the ceiling the interpretable model is compared against. For an end user, the score is only honest if it is a *probability*: raw classifier scores are typically miscalibrated, and post-hoc calibration, such as fitting a sigmoid on held-out validation data (Platt scaling), corrects this (Niculescu-Mizil and Caruana 2005). Under class imbalance, evaluation uses the area under the precision–recall curve rather than accuracy. A budget-shaped precision completes the picture (how many of the N alerts a user can absorb per day were material), because that budget is exactly the alert-fatigue cost the triage exists to reduce.

For InvestiGator: triage adds a *trained* severity score on top of the transparent pipeline, ranked and gated per day, explained by its additive contributions, calibrated on validation data, and always compared against a volatility-only baseline before it is allowed to matter.

2.10 Existing Tools for the Retail Investor

What can a retail investor already use, and what is missing?

Three kinds of tool are common today, and each leaves a gap. *Price alerts*, in every brokerage app, fire when a price or percentage threshold is crossed; they ride the same attention behaviour InvestiGator targets (Barber and Odean 2008), but a fixed threshold ignores each stock's own volatility (firing constantly on volatile names, rarely on calm ones) and says only *that* a level was crossed, never *why*. *News and sentiment apps* aggregate headlines and attach a sentiment score, building on the link between tone and markets (Tetlock 2007) and on finance lexicons (Loughran and McDonald 2011); but a single polarity is thin, often proprietary, and rarely connects a headline to concrete historical precedents and their outcomes. *Robo-advisors*, the most studied retail-fintech tool, genuinely help by improving diversification and curbing biases (Cardillo and Chiappini 2024; D'Acunto, Prabhala, and Rossi 2019), but they *act for* the investor with proprietary logic, the opposite of an explanation-first tool that leaves the decision to the user. Table 2.4 positions all three against InvestiGator.

Table 2.4: Existing retail tools versus the system proposed in this work.

Tool	**What it does**	**Explains why?**	**Shows precedents?**	**Acts for user?**
Brokerage price alert	Fires on a fixed price / % threshold	No	No	No
News / sentiment app (Tetlock 2007)	Aggregates news, scores sentiment	Partly (opaque score)	No	No
Robo-advisor (D'Acunto, Prabhala, and Rossi 2019)	Allocates and rebalances a portfolio	Rarely (proprietary)	No	Yes
This work (InvestiGator)	Explains events, shows evidence	Yes, by design	Yes (measured impact)	No

For InvestiGator: each existing tool covers one corner, alerting, news, or allocation, but none combines volatility-aware detection, transparent explanation and concrete historical evidence in one tool that informs rather than decides. That gap is the niche this work fills.

2.11 Comparative Analysis and Positioning

So what does InvestiGator choose, and why?

Table 2.5 gathers the choices against their alternatives. The rule throughout is *defensible simplicity* (Rudin 2019): when two options perform comparably, prefer the more transparent one, because the system must be understood and acted upon by a non-expert.

Table 2.5: Component choices in this work versus alternatives.

Component	Chosen	Alternative	Why
Anomaly detection	z-score (rolling)	Isolation Forest, deep detectors	Transparency; low-dimensional signal
News retrieval	Sentence-BERT + cosine	Lexicons, word2vec, LLMs	Contextual yet cheap and reproducible
Impact measurement	Event-study windows	—	Standard, reproducible, no forecasting
Explanation	Transparent design + precedents	LIME / SHAP only	Faithful by construction

2.12 Chapter Conclusions

None of these building blocks is new. What is scarce, and what this work builds, is their disciplined integration into a single explainable system for the retail investor: a transparent detector, semantic retrieval paired with event-study impact, and explanations that are faithful by construction. The next chapter turns these choices into a method.

Chapter 3

Methods and Materials

This chapter answers two questions: how is the system built, and how is it judged? It covers the approach, the data (in a reproducible data card), the techniques behind each component, the responsible-AI commitments, and the evaluation design, which was fixed before any experiment was run. One idea runs through all of it: keep things as simple as they can defensibly be, so that when two options perform about the same, the more transparent one wins. InvestiGator itself is described in Chapter 4; the experiments are reported in Chapter 5.

3.1 Approach and Methodology

How was the system built? Incrementally, end to end. Rather than finishing each component in isolation, a thin slice was built and proven first, from data acquisition to a delivered alert, so that integration risk surfaced early. Each component then started in its simplest defensible form and grew only where the extra complexity earned its place. This is what Artificial Intelligence *engineering* means here: the building blocks already exist in the literature (Chapter 2), so the contribution is their disciplined integration, application and critical evaluation, not a new algorithm.

Two habits follow. First, every component is defined by its *responsibility* and its *interface*, not by a particular implementation, which keeps it testable and swappable (one embedding model for another, say). Second, the project's non-negotiable constraints, only free and reproducible resources, the US market, transparency, and no prediction or trading, filter the design space before any performance question.

3.2 Datasets and Data Sources

What data does the system use? Two clearly separated layers that share one schema, so they are directly comparable (Figure 3.1): a **historical** layer, built offline, that supplies the precedents, and a **live** layer that supplies real-time prices and news. The split is deliberate: the historical layer must be rich and stable, the live layer timely and cheap.

3.2.1 Historical Layer: the FNSPID Corpus

The intended primary source for the historical layer is the FNSPID dataset (Dong, Fan, and Peng 2024), which provides financial news time-aligned with daily prices for thousands of US companies. That alignment is exactly what is needed to relate news to subsequent price impact without custom scraping. The dataset's news component is large (on the order of

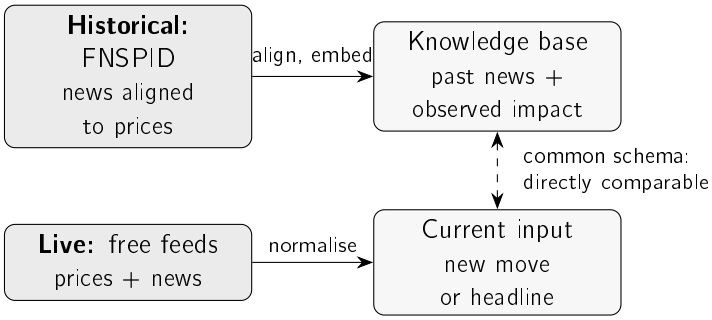

Figure 3.1: The two data layers. The historical layer turns the FNSPID corpus into a knowledge base of past news and its observed price impact; the live layer normalises real-time prices and news into the same fields, so a current event can be compared directly against the precedents.

tens of gigabytes), which directly motivates the streaming, filtered ingestion described below. Table 3.1 records the data card: the choices that make the historical subset reproducible.

Table 3.1: Data card for the *designed* historical layer (FNSPID).

Item	**Value**
Source	FNSPID (Dong, Fan, and Peng 2024) (financial news time-aligned with prices)
Licence	Creative Commons BY-SA 4.0 (attribution required)
Fields used	date, ticker, headline (mapped to a common schema)
Tickers (15)	AAPL, MSFT, AMZN, GOOGL, NVDA, TSLA, META, JPM, BAC, XOM, CVX, JNJ, PFE, WMT, KO
Sectors (5)	Technology, banking, energy, health, consumer staples
Time window	2018–2023 (daily granularity)
Prices	Daily closing prices for the same tickers
Governance	Large data not versioned (recreated by script); only small samples committed

Note: this card documents the FNSPID-based historical layer. The multi-year subset *was* built by the streaming pipeline and drives the materiality-triage study of Chapter 5 (79,753 examples, 2018–2023; fourteen of the fifteen tickers, as the corpus indexes Meta under its former symbol). The retrieval study is run on the real recent-news evaluation corpus described in Section 3.2.3 (3,714 headlines for the same tickers, built through the identical process); rebuilding the retrieval knowledge base from the multi-year corpus remains future work.

The fifteen large-cap tickers span five sectors, which makes the cross-sector evaluation of Chapter 5 meaningful while keeping the corpus tractable; the 2018–2023 window (the dataset's extent) is recent yet long enough to hold precedents. These are documented choices, not fixed ones: a different ticker set or window can be substituted without changing the method.

3.2.2 Preprocessing and Event Alignment

How is a news item turned into a historical case? In three steps. First, the news file is read in a stream and filtered to the chosen tickers and window, so only the relevant subset is materialised. Second, each item is normalised to the common schema (date,

ticker, headline). Third, each item is aligned to the market and its impact measured. The alignment rule matters for honesty: the event day is the first trading day *on or after* the news date, and impact is measured from that day's *close* onward. Measuring from the close, not the open, avoids crediting a move already in the opening price, and measuring strictly forward enforces the no-lookahead guarantee of Section 3.6. Items with no aligned prices, or whose window runs past the data, are dropped rather than imputed. Algorithm 3.1 states the build.

Algorithm 3.1 Knowledge-base construction with event alignment.

Require: news items $\{(d_i, c_i, h_i)\}$; daily close prices P_c per ticker; horizons $H = \{1, 3, 5\}$
Ensure: knowledge base $\mathcal{K}$ of cases $(d, c, h, \mathbf{e}, \text{impact})$

$\mathcal{K} \leftarrow \emptyset$
for each news item (d_i, c_i, h_i) **do**
  $e_i \leftarrow$ first trading day $\geq d_i$ in P_{c_i} ▷ event alignment
  if e_i exists **then**
    $\mathbf{e}_i \leftarrow \text{embed}(h_i)$ ▷ Sentence-BERT
    for $\tau \in H$ **do**
      $\text{impact}_i[\tau] \leftarrow$ return from close(e_i) to close$(e_i{+}\tau)$, or n/a if unavailable
    end for
    $\mathcal{K} \leftarrow \mathcal{K} \cup \{(d_i, c_i, h_i, \mathbf{e}_i, \text{impact}_i)\}$
  end if
end for
return $\mathcal{K}$

To make the abstract concrete, Figure 3.2 follows *one real headline* through every stage of the system: the raw record as retrieved, the cleaning and no-lookahead alignment, the representation step where the "AI" actually happens (the headline becomes a 384-dimensional embedding, the event becomes a small vector of market-context features, and the outcome becomes a binary materiality label), and the two things those representations are finally used and measured for. Every value shown is genuine, taken from the dataset for Nvidia on 10 May 2018.

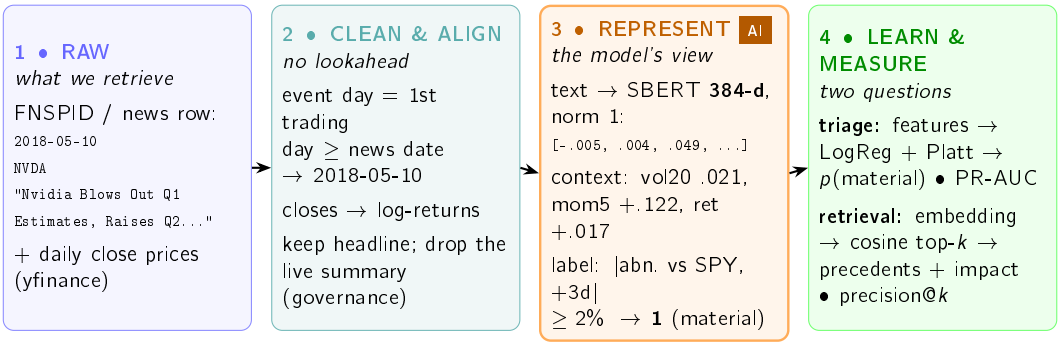

Figure 3.2: One real headline (Nvidia, 10 May 2018) travelling through every stage. It is retrieved as a plain row plus prices (1), cleaned and aligned to the first trading day with impact measured strictly forward (2), then *represented* for the models (3): the text becomes a 384-dimensional Sentence-BERT embedding, the market context becomes a handful of features, and the observed outcome becomes a binary materiality label. Those representations feed the two things the system measures (4): a calibrated triage probability and precedent retrieval. Every value is genuine, read from the dataset.

3.2.3 Live Layer and Evaluation Dataset

The live layer provides real-time prices from a free market-data service (with a free fallback) and news from a free company-news service and RSS feeds. Free news is deliberately *not* used to build the historical layer, because free tiers cover only a short, recent, delayed window; but it is fine as the live trigger and as a source of *real* news for a first evaluation. For that, a set of 3,714 real headlines for the fifteen tickers was collected from the free company-news service (the most recent months available), and is used for the retrieval study in Chapter 5. This corpus is real but recent, which is why the full multi-year FNSPID build remains the richer source, identified as future work.

3.2.4 Data Governance and Attribution

Two governance rules apply to both layers. First, large data and models are never versioned: they are recreated by the ingestion process, and only small illustrative samples are kept under version control. Second, third-party news text is not republished: the dataset is cited and attributed as required by its Creative Commons BY-SA 4.0 licence (Dong, Fan, and Peng 2024), and only minimal headline excerpts appear in this document.

3.3 Models and Techniques

Three techniques do the work: a detector, a retrieval-and-impact engine, and an explanation step. Each is introduced by what it is *for*, then how it works.

3.3.1 Statistical Anomaly Detection

What it is for: to flag a daily move that is unusual *for that stock*, not just large in absolute terms. *How it works:* let r_t be the day's (log-)return and μ_t, σ_t the mean and standard deviation of returns over a trailing window of w days ending strictly before t. The day is flagged when the absolute z-score exceeds a threshold k:

$$z_t = \frac{r_t - \mu_t}{\sigma_t}, \qquad \text{anomaly} \iff |z_t| > k. \tag{3.1}$$

This is the statistical family of detectors (Chandola, Banerjee, and Kumar 2009), chosen for transparency: the investor can be shown exactly how many standard deviations the move was. More expressive detectors exist, Isolation Forest (F. T. Liu, Ting, and Zhou 2008) or deep models (Pang et al. 2021), but their opacity is not warranted for a single return series explained to a non-expert (Chapter 2). The window w and threshold k are explicit choices, examined by ablation in Chapter 5. Algorithm 3.2 states the rule; the window for day t uses only earlier days, so no future information enters.

A small example shows why this matters. Two stocks both fall 3.2% today. One has been calm ($\sigma \approx 0.40\%$), the other volatile ($\sigma \approx 1.50\%$). The same move is a clear anomaly for the calm stock ($z \approx -8.1$) but an ordinary day for the volatile one ($z \approx -2.2$), as Table 3.2 shows. A fixed percentage threshold would flag both; the z-score does not, which is exactly what we want.

Algorithm 3.2 Rolling z-score anomaly detection (no lookahead).

Require: return series $r_1, \ldots, r_T$; window w; threshold k
Ensure: flag a_t for each day, with the quantities behind it

for $t \leftarrow w + 1$ **to** T **do**
  $W \leftarrow (r_{t-w}, \ldots, r_{t-1})$ ▷ strictly earlier days
  $\mu_t \leftarrow \text{mean}(W)$; $\sigma_t \leftarrow \text{std}(W)$
  $z_t \leftarrow (r_t - \mu_t) / \sigma_t$
  $a_t \leftarrow (|z_t| > k)$
end for
return for each flagged t, the tuple $(z_t, \mu_t, \sigma_t, w, k)$ ▷ exposed to the explanation engine

Table 3.2: Worked example: the same −3.2% move on a calm and a volatile stock.

Quantity	Calm stock	Volatile stock
Recent mean μ	0.04%	0.10%
Recent standard deviation σ	0.40%	1.50%
Today's return r	−3.2%	−3.2%
z-score $(r - \mu)/\sigma$	**−8.1**	**−2.2**
Flagged at $k = 3$?	yes	no

3.3.2 Semantic Retrieval and Impact Measurement

What it is for: given a new headline, find the most similar past headlines and show what happened to their prices. This news–market correlation engine is the heart of the work, and it combines two techniques. *Retrieval:* each headline becomes a sentence embedding with Sentence-BERT (Reimers and Gurevych 2019), chosen (Section 2.5) because it captures meaning yet yields directly comparable vectors, so the nearest past items are found cheaply by cosine similarity. *Impact:* for each precedent, the cumulative return is measured over fixed post-event windows (+1, +3 and +5 trading days), in the manner of an event study (Brown and Warner 1985; Fama et al. 1969). Both the similarity metric and the windows are explicit, documented choices, and together they are the methodological contribution. Impact is measured strictly after the event, so it is an outcome, not a forecast. Algorithm 3.3 states the retrieve-and-reuse step (the case-based-reasoning core of Section 2.8).

Algorithm 3.3 Precedent retrieval and impact reporting (news trigger).

Require: headline h; knowledge base $\mathcal{K} = \{(d_j, c_j, h_j, \mathbf{e}_j, \text{impact}_j)\}$; neighbours k; horizon τ
Ensure: top-k precedents with similarity and observed impact, and their mean impact

$\mathbf{q} \leftarrow \text{embed}(h)$ ▷ same Sentence-BERT model as the KB
for each case $j \in \mathcal{K}$ **do**
  $s_j \leftarrow \cos(\mathbf{q}, \mathbf{e}_j)$ ▷ cosine over L2-normalised vectors
end for
$S \leftarrow$ indices of the k largest s_j
return $\{(d_j, c_j, h_j, s_j, \text{impact}_j[\tau]) : j \in S\}$ and $\text{mean}_{j \in S}\, \text{impact}_j[\tau]$

Two details of the embedding step deserve to be explicit, because each answers a question an examiner would rightly ask. The first is what embed(h) actually computes. Sentence-BERT is a *bi-encoder*: a pre-trained transformer reads the headline once and outputs one contextual vector per token, and the sentence embedding is the mean of those token vectors (mean pooling), scaled to unit length:

$$\mathbf{e} \;=\; \frac{1}{n}\sum_{i=1}^{n}\mathbf{t}_i, \qquad \hat{\mathbf{e}} \;=\; \frac{\mathbf{e}}{\|\mathbf{e}\|_2}, \tag{3.2}$$

where $\mathbf{t}_1, \ldots, \mathbf{t}_n$ are the token vectors of the final transformer layer. The deployed encoder (MiniLM) is a distilled six-layer transformer producing 384-dimensional embeddings, small enough for free infrastructure. The bi-encoder design is what makes a knowledge base practical: every stored case is embedded *once*, and a new headline costs a single forward pass plus cheap vector comparisons. The more accurate cross-encoder alternative, which re-reads query and candidate together, would need one full transformer pass per (query, case) pair, which at the scale of Chapter 5 means tens of thousands of passes for every incoming headline. The second detail is the similarity itself. For embeddings $\mathbf{q}$ and $\mathbf{e}$,

$$\cos(\mathbf{q},\mathbf{e}) \;=\; \frac{\mathbf{q}\cdot\mathbf{e}}{\|\mathbf{q}\|_2\,\|\mathbf{e}\|_2}, \qquad \|\hat{\mathbf{q}}-\hat{\mathbf{e}}\|_2^2 \;=\; 2-2\cos(\hat{\mathbf{q}},\hat{\mathbf{e}}), \tag{3.3}$$

and the identity on the right settles the "why cosine?" question: on unit-length vectors the cosine equals the dot product, and Euclidean distance is a monotone function of it, so the three natural similarity measures induce exactly the same ranking. The choice of cosine is therefore canonical rather than arbitrary; what matters is the L2 normalisation, which stops longer texts from looking artificially "bigger" than short ones.

The impact itself is read off a short window after the event, as Figure 3.3 shows. The news may arrive on a non-trading day, so the *event day* is the first trading day on or after it; the return is then accumulated from that day's close to the close +1, +3 and +5 trading days later. Anchoring on the close, and looking only forward, is what keeps the measure free of lookahead: it records what happened *after* the event became actionable, never before.

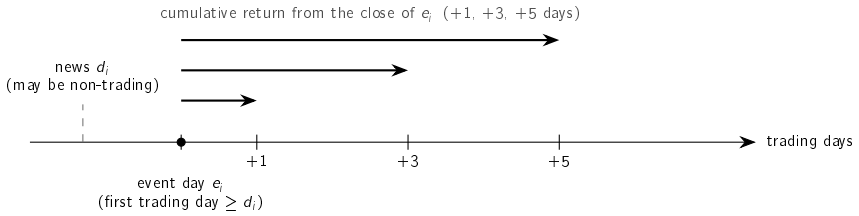

Figure 3.3: How impact is measured. The news date d_i may fall on a non-trading day; the event day e_i is the first trading day on or after it. The return is then accumulated forward from the close of e_i over +1, +3 and +5 trading days. Only post-event days are used, so no future information enters.

Three choices fix *how* impact is measured, each made for transparency as much as rigour:

- **Horizons.** Short windows (+1, +3, +5 days), standard for daily event studies (Brown and Warner 1985); longer windows let unrelated, market-wide moves creep in.
- **Baseline.** The *raw* cumulative return, not a market-adjusted abnormal return, because it is directly observable and needs no estimated market model a non-expert could not check. The cost, stated plainly, is that a raw return mixes the news reaction with any

market-wide move on the same days (the confounding threat examined in Chapter 5); market adjustment is future work.

- **Aggregation.** The headline figure is the *mean* impact across the retrieved precedents, but the individual precedents are always shown too, so the investor sees the spread, not just one number.

One apparent inconsistency is worth resolving here, because two parts of the system measure the aftermath of an event differently, and both are right for their job. The *displayed* impact of a precedent uses the raw return, as chosen above: it is what the investor would actually have experienced, and it can be checked against any public price chart. The *label* of the materiality-triage model (Section 3.3.3) instead uses the market-adjusted difference $|r^{\text{tkr}} - r^{\text{mkt}}|$, because a training target must isolate the ticker-specific move from market-wide swings; otherwise the model would learn to predict the index rather than the consequences of the news. It is the same event-study machinery answering two different questions: "what happened to this stock?" for evidence shown to a person, and "did this stock move for its own reasons?" for a label a model learns from.

A worked example makes it concrete. Figure 3.4 gives the intuition: the embedding places each headline in a space where similar themes fall close together, so a query lands near its matches and far from unrelated news.

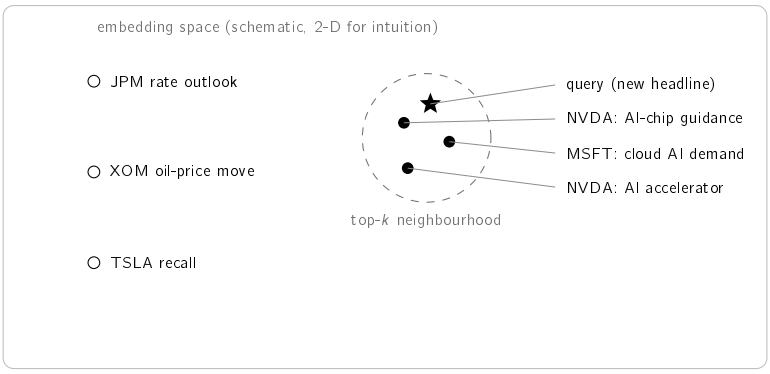

Figure 3.4: Conceptual illustration of semantic retrieval. Each headline becomes a point in a high-dimensional embedding space (drawn in two dimensions for intuition only). Texts on the same theme lie close together, so a query (star) falls near its semantically similar precedents (filled), which form the top-k neighbourhood, and far from unrelated news (hollow). The next table makes this concrete with real numbers.

Table 3.3 runs the step end to end on the small sample knowledge base shipped with the system. To keep it reproducible without downloading a model, it uses a transparent word-overlap encoder (the deployed system uses Sentence-BERT, evaluated in Chapter 5). For the query *"Nvidia demand surges on AI chip orders"*, the three nearest precedents are all AI-themed and not all from the same company: a Microsoft cloud-AI item is retrieved alongside two Nvidia items, showing the *cross-ticker* generalisation the evaluation rests on. Their mean +5-day impact is +6.5%, shown as evidence, with the explicit caveat that it records what followed comparable past news, not a forecast.

Table 3.3: Worked retrieval example on the bundled sample knowledge base. Query: *"Nvidia demand surges on AI chip orders"*; similarities and impacts are real and reproducible.

Similarity	+5-day impact	Ticker / date	Retrieved headline (excerpt)
0.60	+3.5%	NVDA, 2023-05-25	Nvidia guidance surges on soaring demand for AI data-centre chips
0.38	+10.9%	MSFT, 2023-04-25	Microsoft cloud growth accelerates on AI demand
0.38	+4.9%	NVDA, 2023-06-13	Nvidia unveils new AI accelerator to extend data-centre lead
Mean +5-day impact across the retrieved precedents: +6.5%			

Note: the mean is computed from the unrounded impacts, so it need not equal the average of the rounded values shown.

3.3.3 Materiality Triage: a Trained Severity Model

What it is for: to rank the incoming news stream, so that the user's limited attention goes to the headlines most likely to matter (Section 2.9), answering RQ4.

The unit is one (headline, ticker, event day) triple, where the event day d is the first trading day on or after the news date, the same alignment rule the knowledge base uses. The label asks a direction-free question by construction: did an abnormal move of unusual size follow? Formally, the example is *material* when $|r^{\text{tkr}}_{(d,\,d+h]} - r^{\text{mkt}}_{(d,\,d+h]}| \geq \tau$. The quantity inside the absolute value is the ticker's return over the h trading days after the event minus the market's (an index proxy): the market-adjusted outcome of Section 2.7. The primary setting is $\tau = 2\%$ at $h = 3$; a sensitivity grid over $\tau \in \{1.5\%, 2\%, 3\%\}$ and $h \in \{1, 3, 5\}$ is reported so the conclusion does not hinge on one threshold. Labels are produced by the system's own event-study code; no manual annotation is involved.

Features respect a strict timing convention, all computable at the close of day d, before the label window opens: the pre-event volatility (standard deviation of the twenty daily log-returns ending the day *before* the event), the five-day momentum to the same point, the event-day return itself (known at the close of d), the headline length, a sector indicator, and the sentence embedding of the headline. A unit test enforces the convention by mutating prices *after* the event window and asserting that no feature changes while the label does.

Table 3.4 makes those columns concrete: it lists every feature, what it is, and its actual value for the worked example of Figure 3.2 (Nvidia, 10 May 2018), together with the target label. This is exactly the data on which the metrics are computed.

Model families span the interpretability–capacity trade-off of Section 2.9: an always-alert floor, a volatility-only logistic regression (the strong naive rule any reviewer would ask about), logistic regressions on context features only, on text features only, and on both together (the main interpretable model), and a gradient-boosted ensemble (Friedman 2001) as the capacity ceiling.

The interpretable member of that family deserves its mathematics in full, because the deployed explanations decompose it term by term. With standardised features $\tilde{x}_1, \ldots, \tilde{x}_m$, the

Table 3.4: The columns the triage model reads, with their real values for one example (Nvidia, 10 May 2018). Rows 1–6 are the features; the last row is the supervised target. The context columns (1–5) feed the logistic regressions and the gradient-boosted ensemble, scored by PR-AUC and precision@5/day; the embedding (6) also feeds precedent retrieval, scored by precision@k. Every feature is computable at the close of day d, before the label window opens.

Column	What it is	Example value
`vol20`	pre-event volatility: sd of the 20 daily log-returns ending at $d-1$	0.021
`mom5`	five-day momentum to $d-1$ (cumulative log-return)	+0.122
`ret_event`	event-day log-return, known at the close of d	+0.017
`headline_len`	number of characters in the headline	55
`sector`	one-hot sector indicator	tech
`embedding`	384-d Sentence-BERT vector of the headline	$[-0.005, 0.004, \ldots]$
`label` *(target)*	$\lvert$abnormal return vs SPY over $(d, d+3]\rvert \geq 2\%$	1 (material)

logistic regression scores an example as

$$u \;=\; b_0 + \sum_{j=1}^{m} w_j \tilde{x}_j, \qquad p_{\text{raw}} \;=\; \sigma(u) \;=\; \frac{1}{1+e^{-u}}, \tag{3.4}$$

so each product $w_j\tilde{x}_j$ is an *exact* additive contribution to the log-odds u: no surrogate model and no approximation, the property that makes the alert's "why" line faithful by construction. Because a discriminative score is not automatically an honest probability, a Platt calibration is fitted on the validation block only,

$$p \;=\; \sigma\big(a\, p_{\text{raw}} + c\big), \tag{3.5}$$

with just two scalars (a, c). The parametric form is a deliberate choice over the non-parametric alternative, isotonic regression: a two-parameter sigmoid cannot overfit a modest validation block, whereas isotonic regression needs substantially more calibration data before its extra flexibility pays off (Niculescu-Mizil and Caruana 2005). An isotonic comparison on the full corpus is noted as future work rather than assumed either way.

Table 3.5 makes the whole computation concrete by decomposing one *real* production decision: a Meta headline sent to the public alert channel on 12 July 2026, scored by the deployed context-only bundle. Reading the table top to bottom is reading Equations (3.4) and (3.5) with numbers in place: elevated recent volatility and the sector indicator push the log-odds up, the near-zero terms barely move it, the sum gives $u = +0.699$, the sigmoid turns it into $p_{\text{raw}} = 0.668$, and the Platt map (fitted values $a = 3.700$, $c = -2.313$) yields the calibrated $p = 0.539$, which clears the deployment gate of 0.5. The message actually delivered to the channel that day read *"Risk estimate (learned triage): 54% … raised by recent volatility (20d) and sector"*: the decomposition reproduces the production number and its dominant factors exactly. For explanations, the embedding dimensions (when the

text variant is used) are grouped into a single "headline content" term so the message stays readable, and every score is delivered with the fixed clause *"triage evidence, not a forecast"*.

Table 3.5: Worked triage example: the deployed context-only model scoring a real alert (META, 12 July 2026, *"Mark Zuckerberg Said Meta's AI Bets 'Haven't Come to Fruition Yet' as Shares Fell 5%"*). Contributions are the exact additive terms $w_j\tilde{x}_j$ of Equation (3.4); the decomposition is reproducible via `scripts/figures/fig_triage_worked.py`.

Term	**Contribution to log-odds**
Intercept b_0	-0.025
Recent volatility (20 d)	$+0.409$
Sector indicator	$+0.303$
Event-day return	$+0.010$
Headline length	$+0.002$
Recent momentum (5 d)	$+0.000$
Log-odds u (sum)	$+\mathbf{0.699}$
Raw probability $\sigma(u)$	0.668
Calibrated $p = \sigma(3.700\, p_{\text{raw}} - 2.313)$	$\mathbf{0.539}$
Deployment gate $p \geq 0.5$	passed (alert sent)

Deployment is deliberately conservative. The score enters the alert pipeline off by default, behind a configuration gate; the deployed variant uses context features only, so it runs on the light software stack, and the alert text names the variant used. Once live, every triage decision is logged, and a post-validation step later labels each matured decision with what actually happened, using the same label rule, yielding live precision and calibration monitoring and fresh training data. This is continual learning with delayed labels, not reinforcement learning: there is no action–environment feedback loop, because the system's alerts do not move the market.

3.3.4 Explanation Techniques

What it is for: to turn the computed objects into a message the investor can check. Explanation is pursued by transparent design (Section 2.3) (Barredo Arrieta et al. 2020; Rudin 2019). Each alert is assembled from three transparent ingredients: the rule that fired (the z-score with its window and threshold); the retrieved precedents with their observed impact; and, optionally, feature attribution over the detector with SHAP (Lundberg and S.-I. Lee 2017). Financial sentiment from a domain model (Araci 2019) is an optional enrichment, used only if it adds defensible value. Because every alert is rendered directly from these objects, the explanation is faithful to the computation by construction, a property tested in Chapter 5.

3.4 Tools and Frameworks

The system is built on the open-source scientific-Python ecosystem: mature, free libraries for numerical computing, market-data access, sentence embeddings and figures. This fits the free-resources constraint and reproducibility, since pinned dependencies and fixed seeds

let the environment and results be recreated elsewhere. The concrete engineering (environment, code organisation, tests) is kept in Appendix A, so this chapter stays a description of *methods*, not software.

3.5 Responsible AI and Ethics

The work is bound by several responsible-AI commitments that follow directly from its purpose and its audience. **No advice and no prediction:** the system performs neither price prediction nor trading recommendation; it surfaces observed past outcomes as evidence and states this explicitly in every alert, so that the investor is informed rather than instructed. **Transparency:** every alert exposes its full reasoning chain, in keeping with the explainability principles of Chapter 2, so that a user can verify rather than merely trust the output. **Honest evidence:** only results that can be reproduced and explained are reported, and no data, metric or citation is fabricated. **Data stewardship:** all sources are used within their free tiers and licences, the FNSPID dataset is attributed as its Creative Commons licence requires (Dong, Fan, and Peng 2024), third-party news text is not republished, and no personal data is processed. **Security:** credentials are never stored in versioned files. Finally, the system is positioned as a decision-support aid for a non-professional investor, not as a substitute for professional advice, a limitation stated plainly rather than hidden.

3.6 Evaluation Methodology

How is the system judged? By a plan fixed before any experiment ran, so the metric cannot be chosen after the fact to flatter the result, a small-scale pre-registration. The results are in Chapter 5; here is the protocol they follow.

3.6.1 Retrieval metric and relevance

Retrieval is evaluated as an information-retrieval task (Section 2.6) with precision@k, the share of the top k precedents that are relevant. Writing Q for the set of query headlines and $R_k(q)$ for the top k items returned for query q,

$$\text{precision@}k = \frac{1}{|Q|}\sum_{q\in Q}\frac{1}{k}\sum_{d\in R_k(q)}\text{rel}(q,d), \qquad \text{rel}(q,d) = \mathbf{1}\big[\text{sector}(d) = \text{sector}(q)\big]. \tag{3.6}$$

Relevance is approximated by *sector agreement*: a precedent counts as relevant if its company is in the query's sector. This is an automatic stand-in for "analogous", used because no labelled set exists; its limits are stated with the results. Two choices make the metric demanding, not flattering. First, a strict *cross-ticker* constraint drops every precedent sharing the query's ticker, so the system cannot win by matching a company to its own past news; it must generalise *across* companies in a sector. Second, k is small (the headline figure uses $k = 5$), reflecting that an alert shows only a few precedents.

3.6.2 Baselines

A single number means little alone, so semantic retrieval is compared against three baselines, each controlling for a rival explanation. **Random** draws k precedents at random; its precision is the sector base rate, the no-skill floor. **Recency** returns the newest items, testing whether

simply surfacing fresh news would do as well. **Lexical** ranks by word overlap under the same cosine, so the gap to the embedding model isolates the value of *meaning* over surface words. Each comparison is repeated over five random seeds (mean with standard deviation), and two Sentence-BERT variants are tried, so the result reflects semantic retrieval in general, not one lucky draw or one model.

3.6.3 Anomaly-detection protocol

The anomaly detector is judged primarily by an argument that needs *no label*: a good universal detector should fire at a comparable rate across instruments of very different volatility, whereas a volatility-blind rule cannot. Consistency is therefore measured as the range (maximum minus minimum) of per-ticker firing rates: the smaller the range, the more uniform the detector. Only secondarily is the detector scored against an explicit *extreme-move* proxy (a daily move at or beyond the 99th percentile of that ticker's own returns), via precision, recall and F_1; because this proxy is itself volatility-relative, it is treated as supporting rather than primary evidence. An ablation over the estimation-window length completes the picture by exposing the detector's sensitivity to its one main parameter. Finally, the transparent rule is challenged by a *learned* detector on equal terms: an Isolation Forest (F. T. Liu, Ting, and Zhou 2008) trained causally on the same information (each day's return and the volatility of the twenty preceding days, fitted only on an initial past segment) is scored on the same evaluation region, so the choice of a statistical rule over a learned one is tested, not assumed. Two extensions harden that comparison further. The Local Outlier Factor (Breunig et al. 2000), cited in Chapter 2 as the density-based alternative, is evaluated under the identical causal protocol, so the statistical rule faces *two* learned detectors rather than one. And the rolling volatility estimate is challenged by an exponentially weighted variant (RiskMetrics-style, $\lambda = 0.94$), the first empirical step toward the GARCH family of Section 2.2, so the natural "why not GARCH?" question receives evidence rather than opinion.

3.6.4 Triage protocol

The materiality model of Section 3.3.3 is evaluated under a strict temporal protocol. Examples are split into training, validation and test blocks chronologically, by *unique event day* (70/15/15), so that all headlines from one day land in the same block, and an embargo of several trading days is removed after each boundary so that no label window can straddle two blocks. Calibration is fitted on the validation block only; the test block is touched once, to report. The primary metric is the area under the precision–recall curve, whose floor is the test prevalence (an always-alert strategy); the area under the ROC curve and the Brier score complete the picture, and a product-shaped metric, precision within a budget of N alerts per day, measures what the user actually experiences. The metrics themselves are standard but worth fixing precisely. Sweeping a threshold s over the scores traces $(\text{recall}(s), \text{precision}(s))$; PR-AUC is the area under that curve, and a scorer with no skill earns exactly the prevalence π, which anchors every comparison. ROC-AUC is the probability that a random positive outscores a random negative; it is reported for completeness but not used as primary, because it rewards ranking the abundant easy negatives and so flatters a triage model whose user only ever sees the top of the ranking. Calibration is measured by the Brier score,

$$\text{Brier} \;=\; \frac{1}{N}\sum_{i=1}^{N}\left(p_i - y_i\right)^2, \tag{3.7}$$

the mean squared gap between the stated probability p_i and the outcome $y_i \in \{0, 1\}$: it is low only when the probabilities can be taken at face value, which is exactly what a user reading "57%" deserves. The decisive comparison is against the volatility-only logistic regression: the learned model is claimed useful only where it beats that baseline, and the feature ablations (context only, text only, both) attribute any gain to its source. Sensitivity over the $\tau \times h$ label grid is reported alongside. Two further guarantees are pre-committed. Training is deterministic given the seed, and rerunning it reproduces the stored artefacts exactly. And the result is reported whichever way it falls: if the trained model fails to beat the volatility baseline on a given corpus, that finding stands as stated.

3.6.5 Explanation, scope, and rigour guarantees

The explanation engine is assessed for *fidelity*: whether the alert text reflects the system's actual computation. This holds by construction, because each alert is rendered directly from the computed objects. Its *usefulness* to a real investor is a separate matter, which would need a human study not yet conducted, and is reported as a limitation rather than a claim. Because the system performs neither price prediction nor trading, profit-based metrics are out of scope by design, not omitted by convenience. Underpinning every experiment are three guarantees. First, **no lookahead**: any feature computed at an instant uses only information available at or before that instant, and historical impact uses only post-event windows. Second, **reproducibility**: random seeds are fixed, dependencies are pinned, and every data and results figure is produced by a versioned script. Third, **honesty**: results are reported with their caveats, and modest but genuine findings are preferred to inflated ones.

3.7 Chapter Conclusions

This chapter set the method: defensible simplicity and incremental delivery; a two-layer data architecture in a reproducible data card; the techniques for detection, retrieval, impact and explanation, each introduced by its purpose; and an evaluation fixed in advance. The next chapter assembles these into InvestiGator.

Chapter 4

InvestiGator: An Explainable Financial-Alert System for Retail Investors

This chapter answers five questions, in order: *What is the system, in one picture? What objects does it work with? What are its parts, and who does what? How does data flow through it? And how does it decide?* Chapter 3 argued for the individual methods; here they fit together into one working system, InvestiGator.

4.1 Overview

What is the system, in one picture?

InvestiGator watches the US equity market for a retail investor and raises an alert when one of two independent triggers fires: an abrupt price movement, or a relevant incoming news item. Every alert carries a full, traceable explanation, not a bare notification. The system is a pipeline of single-responsibility components linked by a common data schema, so the live and historical layers are directly comparable. Figure 4.1 is the one-picture view: two data sources and the knowledge base feed two engines, whose evidence converges on one explanation step before delivery.

Three engineering principles keep the system testable and defensible. First, **a thin slice was built end to end before breadth**: the market trigger worked from price to delivered alert before the heavier correlation engine was added, which controlled integration risk. Second, **pure logic is separated from input/output**, so the numerical core runs and is tested offline, without a network or the model stack. Third, **components are programmed to interfaces**: the embedding step has two interchangeable implementations (a dependency-free lexical baseline and Sentence-BERT), which is what makes the model comparison in Chapter 5 fair. Table 4.1 gathers the principal design decisions; in a work of this kind, that engineering judgement *is* the contribution.

4.2 The Data Model

What objects does the system work with, and how do they relate?

The central object is a *case*: one past news item together with what happened to its price afterwards. Everything else exists to build cases, store them, and retrieve them. Figure 4.2 shows the objects and their relationships.

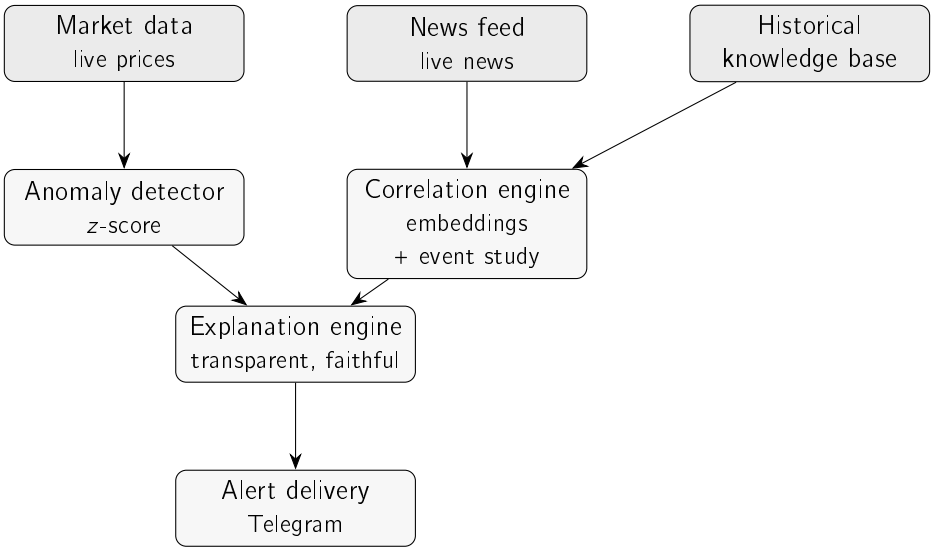

Figure 4.1: Architecture of InvestiGator. The live layer (market data, news feed) and the historical knowledge base feed two engines (the anomaly detector and the correlation engine) whose evidence converges on the explanation engine before an alert is delivered through Telegram. The market-movement trigger follows the left path; the news trigger the right.

Two points make the model easy to hold in mind. First, the *shared schema*: a live **NewsItem** and a historical **NewsRecord** carry the same date, ticker and headline, so a fresh headline can be compared directly against the stored cases. A NewsRecord is simply a NewsItem that has been enriched with its headline *embedding* and its measured *impact* at +1, +3 and +5 trading days. Second, the *knowledge base* is just a collection of cases; asking it for precedents returns the top-k most similar cases with their similarity scores. The market side is simpler still: the detector emits one **AnomalyResult** holding the move and every quantity behind it (z, μ, σ, window, threshold). These few objects are all the explanation engine ever needs.

4.3 Components and Responsibilities

What are the parts, and who does what?

InvestiGator is one package per component, each with a single responsibility and a small interface, which is what lets the numerical core be tested offline and one implementation be swapped for another. Table 4.2 lists them with their one job and the data they turn into.

4.4 Data Flow

How does data move through the system, end to end?

Figure 4.3 traces the whole pipeline as three lanes that converge. The *offline* lane (top) is run once: FNSPID news is aligned to prices and embedded into the knowledge base (Dong, Fan, and Peng 2024). The two live triggers run online. The *news* lane (middle) embeds an incoming headline, finds its nearest cases in the knowledge base, and attaches their measured impact. The *market* lane (bottom) turns live prices into a z-score and fires only

Table 4.1: Principal design decisions in InvestiGator and their rationale.

Decision	Choice	Rationale
Detection method	Rolling z-score	Transparent; every alert is explainable to a non-expert
News representation	Sentence-BERT embeddings	Contextual meaning, yet cheap and reproducible to compare
Impact measurement	Event-study windows	Standard, reproducible; characterises an outcome, not a forecast
Explanation strategy	Transparent design + precedents	Faithful by construction, not a post-hoc approximation
Data architecture	Shared schema, two layers	Live and historical evidence are directly comparable
Component coupling	Programmed to interfaces	Testable offline; enables fair model substitution
Delivery channel	Telegram bot	Free, ubiquitous, no bespoke client needed

when it crosses the threshold. Both lanes hand their evidence to the explanation engine, which delivers one alert through Telegram.

One detail in the offline build matters for honesty: each news item is aligned to the first trading day on or after its date, and impact is measured only from that day's close onward, so a jump already in the opening price is never credited to the system. Building the full multi-year corpus is a long one-off job, so the evaluation in Chapter 5 uses a knowledge base built from real recent news through the identical process, with the full build left as future work.

4.5 Decision Logic

How does the system actually decide? Each trigger applies one simple, transparent rule.

Market trigger. Flag the day when the absolute z-score crosses the threshold, $|z_t| > k$, with z_t computed from the trailing window as in Equation 3.1. The detector returns not just the yes/no decision but every quantity behind it (z, μ, σ, window, threshold), which is exactly what makes the explanation faithful. The same rule is applied across whole price series in the evaluation, so the experiments use the production logic.

News trigger. Embed the incoming headline, rank the knowledge base by cosine similarity, and take the top-k cases. For each, report the observed impact at +1, +3 and +5 days, measured from the event-day close, and show their mean as evidence. Nothing here forecasts: the figures are what *did* happen after comparable past news, and the knowledge base is consulted but never decides.

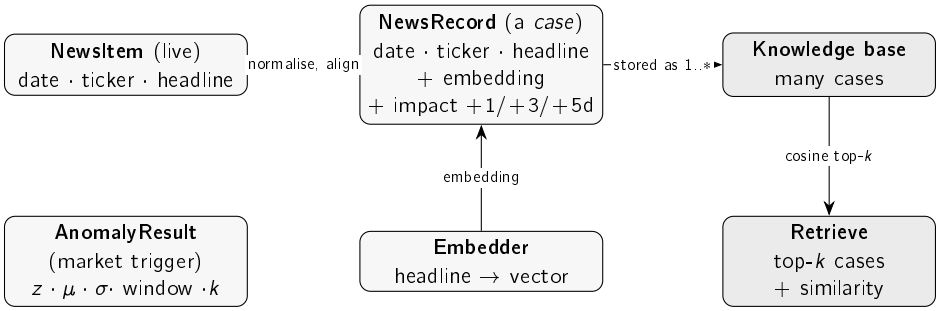

Figure 4.2: The objects InvestiGator works with. A live **NewsItem** and a historical **NewsRecord** share the same schema (date, ticker, headline); a NewsRecord is a *case*, adding the headline's embedding and its measured post-event impact. The **Embedder** produces those vectors; the **Knowledge base** holds many cases; a query headline retrieves the top-k most similar cases with their similarity scores. The market trigger produces a separate **AnomalyResult**.

Table 4.2: InvestiGator's components, each with a single responsibility.

Component	**Responsibility**	**Input → output**
Market data	Fetch prices, compute log-returns	ticker → daily returns
News feed	Fetch and normalise live news	ticker → NewsItem(s)
Embedder	Turn a headline into a vector	headline → embedding
Knowledge base	Hold past cases; retrieve the nearest	headline → top-k cases
Anomaly detector	Flag volatility-relative moves	returns → AnomalyResult
Correlation engine	Cosine similarity + event-study impact	headline, KB → precedents + impact
Triage model	Learned severity: calibrated P(abnormal move follows)	headline + context → probability + contributions
Explanation engine	Render a faithful alert from the objects	objects → alert text
Alert delivery	Send the alert to the investor	alert text → Telegram
Orchestration	Run each trigger end to end	trigger → delivered alert

Learned severity (triage). On top of the two transparent rules sits the optional trained component of Section 3.3.3: each incoming news item is scored with the calibrated probability that an abnormal move follows, and the alert can be *gated*, sent only when the score clears a configured threshold, and *annotated* with the score and its top additive contributions. The gate ships off by default and fails open: with no model file, or too little price history to compute the context features, the pipeline behaves exactly as before. The deployed variant uses context features only, so it runs without the heavy embedding stack, and the alert line always names the evidence for what it is: triage over historical cases, not a forecast.

4.6 Explanation and Delivery

How is the alert explained, and how does it reach the investor?

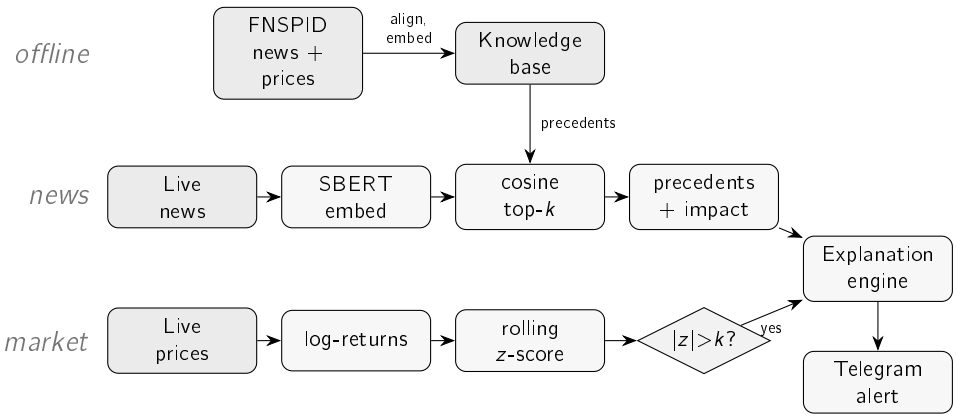

Figure 4.3: The end-to-end flow of InvestiGator as one connected pipeline. *Offline* (top), FNSPID news is aligned to prices and embedded into the knowledge base. The *news trigger* (middle) embeds an incoming headline, finds its nearest precedents in the knowledge base and attaches their observed impact; the *market trigger* (bottom) turns live prices into a z-score and fires only when it exceeds the threshold. Both paths converge on the explanation engine, which delivers one explained alert through Telegram. This is the simplified view; the complete pipeline, with every decision gate and its live configuration value, is Figure A.1 in Appendix A.

The explanation engine renders every alert directly from the objects above, so the text cannot diverge from the computation. For the market trigger it states the move, the z-score, the threshold and window, and the recent mean and standard deviation, and it puts the z-score in plain language (how many times a typical day's move it represents) so a non-expert can read it. For the news trigger it lists the retrieved precedents (date, ticker, similarity, measured impact, headline) and their average impact, and always ends with an explicit disclaimer that these are observed past outcomes, not a prediction.

Delivery is over Telegram, chosen for its free, ubiquitous bot interface. A real alert was delivered during testing; Figure 4.4 shows the format the investor receives, with the whole reasoning chain in one message. In the deployed system the message layout was later compacted for readability (the strongest fact first, the method in a short footnote, and the range of precedent outcomes shown alongside their average); the fields carried are exactly the ones above, so the faithfulness argument is unchanged.

The same reasoning is also published as a live dashboard (Figure 4.5): a "market now" strip summarises the whole watchlist's day at a glance (companies in one sector often move together, so the context matters); below it, one selected company at a time is shown with a large price chart on which every detected event (market anomalies and material news — exactly the alerts delivered to the Telegram channel, read from one shared, versioned record and never recomputed) is marked and can be hovered for the full alert text; the same events are listed in a table below, and a compact line reports the background risk from the trained triage model (Section 3.3.3), computed every day even without a fresh headline. Everything else (method, evaluation, citation) lives on a separate secondary view, keeping the main surface read-only and minimal.

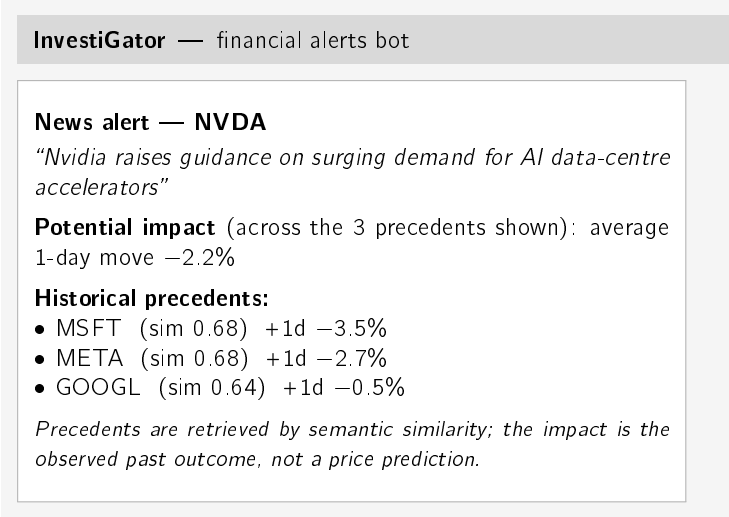

Figure 4.4: An InvestiGator news alert as delivered through Telegram. A single message carries the full, traceable reasoning chain: the detected event, the average observed impact of analogous past events, the individual precedents with their similarity scores and measured impact, and an explicit no-prediction disclaimer. The figures shown are a representative, rounded illustration; the full real end-to-end output appears in Chapter 5 (Case Study 3).

4.7 The Life of One Alert

What does the whole pipeline do, concretely, from raw feed to phone? The cleanest answer is to follow one *real* alert through every stage. The case is the same production decision decomposed in Table 3.5: on 12 July 2026 the public channel received an alert for Meta on the headline *"Mark Zuckerberg Said Meta's AI Bets 'Haven't Come to Fruition Yet' as Shares Fell 5%"*. Table 4.3 walks the stages with the values the system actually computed; every row is verifiable in the shared history record the dashboard reads.

The same gates, seen in aggregate, are the system's answer to alert fatigue. Figure 4.6 shows the first five days of live operation: the scan captured 944 distinct relevant headlines across all ten watchlist names, and the channel received 42 alerts, a 22:1 selectivity concentrated on the three names where the evidence cleared every gate. News flowed for every company; alerts happened only where a strong precedent *and* a material triage score coincided. (One honest note: the per-ticker daily cap entered production on 11 July, so the two earliest days show more alerts per name than the steady state.)

4.8 Practical Use and Responsible Design

What does it take to run, and what does responsible use require?

InvestiGator runs as two entry points, one per trigger, each performing a complete cycle. In day-to-day use they run on a schedule: a reference deployment exists in which a free continuous-integration timer repeatedly scans a watchlist through US market hours and delivers any alerts to a Telegram channel, with an interactive dashboard published alongside. Running the system needs only free resources: a market-data source, a company-news or RSS source (an API key kept out of version control), and the knowledge base built once from the corpus. Appendix A gives the exact steps. Real alerts delivered during testing and by

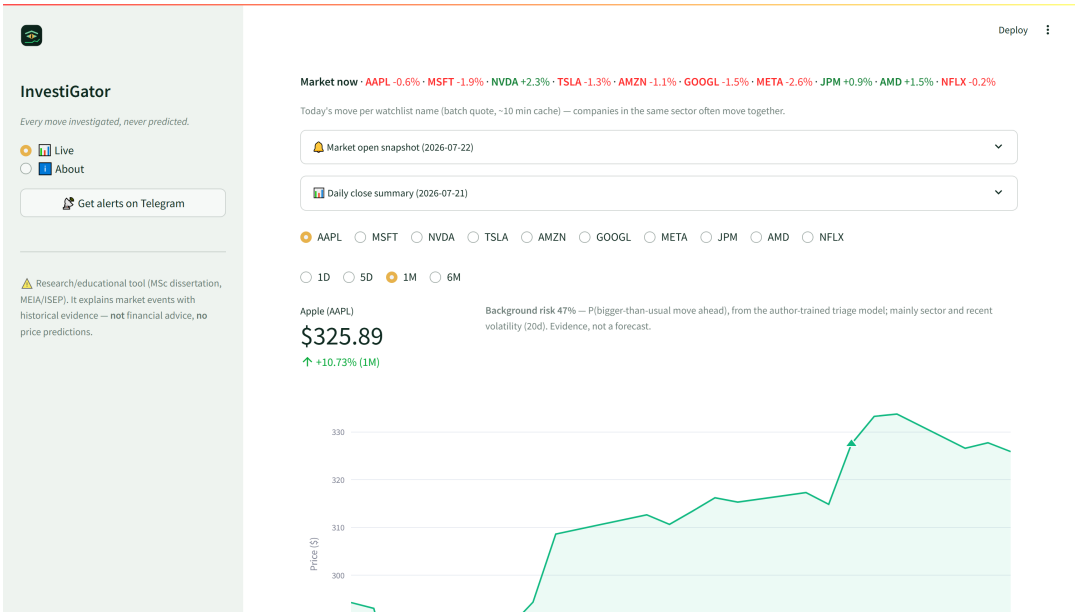

Figure 4.5: The public dashboard (a genuine screenshot, not a mockup): the watchlist-wide "market now" strip, the morning open and previous-close summaries as collapsible notes, one company selected at a time, a large ranged price chart with detected events marked on the curve (hover shows the full alert; the marker here is a real alert from the public channel), and the trained triage model's background risk. Captured from the deployed codebase during live operation.

the scheduled scan show the path from raw data to the investor's phone works end to end. Once live, the triage decisions are also *post-validated*: each logged decision is labelled days later with the outcome that actually followed, giving live precision and calibration monitoring and fresh training data (Section 3.3.3).

Two practical limits are worth stating plainly. The free news tier returns only a recent window, so the knowledge base is shallower than the multi-year build it is designed for (a coverage limit, not a method one). And InvestiGator is a decision-support prototype, not a production service: no user accounts, no persistence beyond the knowledge-base file. These are ordinary steps from prototype to product, recorded openly rather than hidden.

Live operation also taught one engineering lesson the honest way. During the first days of continuous deployment the market path went silent: the free price source blocks the shared network addresses of hosted runners, and with a single source and no fallback, the scan simply received no data. The failure was diagnosed from the system's own records (news alerts kept flowing while not one market summary appeared, which isolated the price feed as the cause) and answered with engineering rather than with a parameter change: daily prices now come through a chain of free providers tried in order, and the real-time quote check, which uses an authenticated and therefore reliable source, was promoted from the optional watcher into every scheduled scan. The episode is reported because it is instructive: a single free data source is a single point of failure, and a system that logs its own behaviour is what makes such a failure diagnosable at all.

A tool aimed at non-experts also raises two responsible-design questions. The first is *alert fatigue*: a tool that fires too often trains the user to ignore it. The market trigger addresses

Table 4.3: The life of one real alert (META, 12 July 2026), stage by stage. Every value is real: the message is preserved verbatim in the shared alert history, and the triage score is decomposed term by term in Table 3.5.

Stage (gate)	**What happened for this alert**
1. Fetch	The scheduled scan pulls the week's company news for each watchlist ticker from the free news API.
2. Relevance filter	The headline names the company ("Meta", "Mark Zuckerberg" are in the ticker's alias list) and matches no market-roundup boilerplate pattern: kept.
3. Capture (living KB)	The headline is embedded (MiniLM, 384-d) and stored as a *pending* case; it matures into a precedent once its own +5-day impact becomes observable.
4. Freshness	Dated within the two-day window: eligible (the same story cannot re-alert for days).
5. Retrieval	Knowledge bases are ranked by cosine with age decay; top three: MSFT 2023-11-07 ($s = 0.46$, +2.70% in 5 d), NVDA 2023-08-02 ($s = 0.51$, −3.87%), NVDA 2023-09-21 ($s = 0.46$, +5.05%).
6. Evidence floor	Best cosine $0.51 \geq 0.45$: strong enough to show. The precedents disagree in direction, so the message carries the explicit *"moved in BOTH directions"* warning.
7. Learned triage	The deployed context-only model scores $p = 0.54 \geq 0.5$ (decomposed in Table 3.5): not gated out.
8. Caps and dedup	Under the two-alerts-per-ticker-per-day cap; not previously sent by any producer (shared-history dedup): kept.
9. Delivery and record	Sent to the public Telegram channel with the fixed no-prediction clause; appended verbatim to the shared history, from which the dashboard marks it on the price chart.

it by firing only on volatility-relative extremes, and the news side is addressed by the triage model, whose severity score exists precisely to rank the stream and gate the least material items (Section 4.5); the evaluation of Chapter 5 therefore includes a precision-within-daily-budget metric that measures this cost directly. The second is *over-reliance*: a confident explanation can be trusted more than its evidence warrants (Section 2.4). The present design mitigates this by showing the individual precedents and their spread rather than a lone number, and by disclaiming prediction on every alert; natural extensions are to de-duplicate near-identical precedents and to flag when a retrieved cluster disagrees in direction.

4.9 Chapter Conclusions

InvestiGator is a two-trigger, two-layer pipeline built from a few clear objects (a *case*, the knowledge base, an anomaly result) and single-responsibility components, with explainability built in by rendering every alert directly from the system's own computation, and an optional trained triage score, off by default and honestly labelled, layered on top. The next chapter puts the core components to the test on real data.

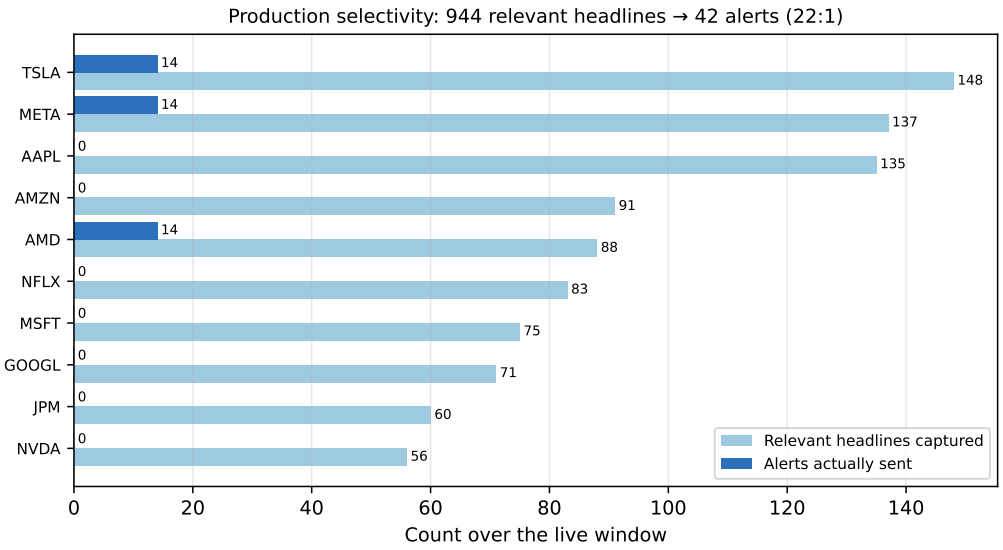

Figure 4.6: Production selectivity over the first five live days: relevant headlines captured (light) versus alerts actually sent (dark), per watchlist ticker. All counts are real, read from the shared data branch; the gap between the two bars is the work of the quality gates of Table 4.3.

Chapter 5

Case Studies

This chapter evaluates InvestiGator through four case studies on real data. The first looks at the anomaly trigger across stocks of very different volatility; the second at the correlation engine and the precedents it retrieves; the third walks through a complete alert and asks whether its explanation is faithful; the fourth trains and tests the materiality-triage model on the multi-year corpus. Every number below comes from a versioned script with a fixed seed, the evaluation followed the plan set out in Chapter 3, and each study states its own limitations.

5.1 Common Experimental Setup

What do the four studies share? Three real datasets are used. For the anomaly trigger, daily closing prices for the fifteen large-capitalisation tickers listed in the data card were retrieved from a free market-data service over a fixed three-year window (June 2023 to June 2026), yielding 750 daily log-returns per ticker. For the correlation engine, 3,714 real financial-news headlines for the same tickers were collected through a free company-news service (the most recent months available on the free tier), spanning five sectors (technology, banking, energy, health and consumer staples). This is the retrieval evaluation corpus, distinct from the designed FNSPID layer documented in the data card of Chapter 3. For the materiality-triage study (Case Study 4), the multi-year FNSPID subset of the data card *was* built, by the streaming pipeline, and aligned to prices: 79,753 (headline, ticker, event-day) examples over 2018–2023. News headlines and prices are normalised to a common schema so that the live and historical layers are directly comparable. The recent corpus is enough for the retrieval experiment; a multi-year impact analysis over the retrieval knowledge base remains future work. The no-lookahead, reproducibility and honesty guarantees of Section 3.6 apply to every study below.

5.2 Case Study 1: Transparent Anomaly Detection on US Equities

Is a rolling z*-score a better universal detector of abrupt moves than a simple fixed-percentage threshold?* **Yes, and clearly so**: the z-score (Chandola, Banerjee, and Kumar 2009) fires at a near-constant rate across stocks of very different volatility, where the fixed rule does not.

The setup is as follows. A "true" anomaly is proxied by a daily move in the upper percentile of *that ticker*'s own returns ($|r| \geq$ the 99th percentile); the baseline is one global threshold

($|r| \geq 3\%$) applied to every ticker. Because that label is itself volatility-relative, it is only supporting evidence; the primary argument, made first below, needs no label at all.

Firing-rate consistency (main argument). The strongest evidence is independent of any label. A good universal detector should fire at a comparable rate across instruments; a volatility-blind threshold cannot. Table 5.1 and Figure 5.1 show that the fixed threshold fires on roughly 1% of days for a calm stock (KO) but on about 35% of days for a volatile one (TSLA, NVDA), whereas the z-score fires at a near-constant rate ($\approx$ 2–3%) for all tickers. The range of firing rates across tickers is more than twenty times tighter for the z-score (0.015 versus 0.344), confirming that normalising by recent volatility makes detection comparable across the market.

Table 5.1: Firing rate across the fifteen tickers (three years of daily data).

Method	Min rate	Max rate	Range
z-score (window 20, ± 3)	0.016	0.031	**0.015**
Fixed threshold ($\lvert r\rvert \geq 3\%$)	0.009	0.353	**0.344**

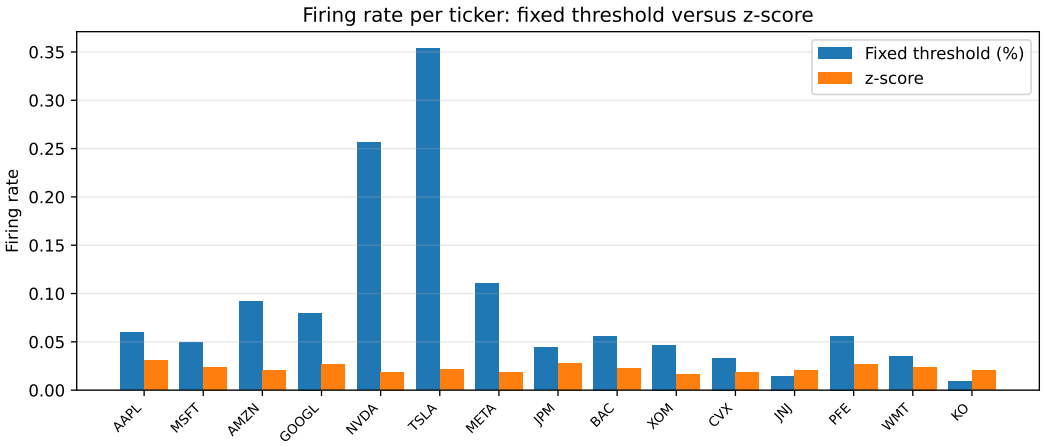

Figure 5.1: Firing rate per ticker. The fixed percentage threshold varies dramatically with each stock's volatility, while the z-score is stable across all tickers.

A worked example. Figure 5.2 shows the detector at work on a single volatile stock (Tesla) over the three-year window. The flagged days are not simply the largest raw moves: they are the moves that are large *relative to the asset's own recent volatility*, which is exactly the behaviour a fixed percentage threshold cannot reproduce. Over this period the detector flags 16 of the 750 trading days (about 2%), consistent with the near-constant firing rate reported above.

To make this concrete, consider the single largest surprise in the window. On 24 October 2024, following its quarterly earnings, Tesla's daily log-return was $+0.198$, a price move of about $+22\%$. Measured against the preceding twenty days, whose mean log-return was -0.92% and standard deviation 2.73%, this is a z-score of $+7.6$, far beyond the threshold $k = 3$ (Table 5.2). The same rule that leaves an ordinary two-percent wobble unflagged identifies this earnings reaction at once, and exposes the quantities that justify the alert:

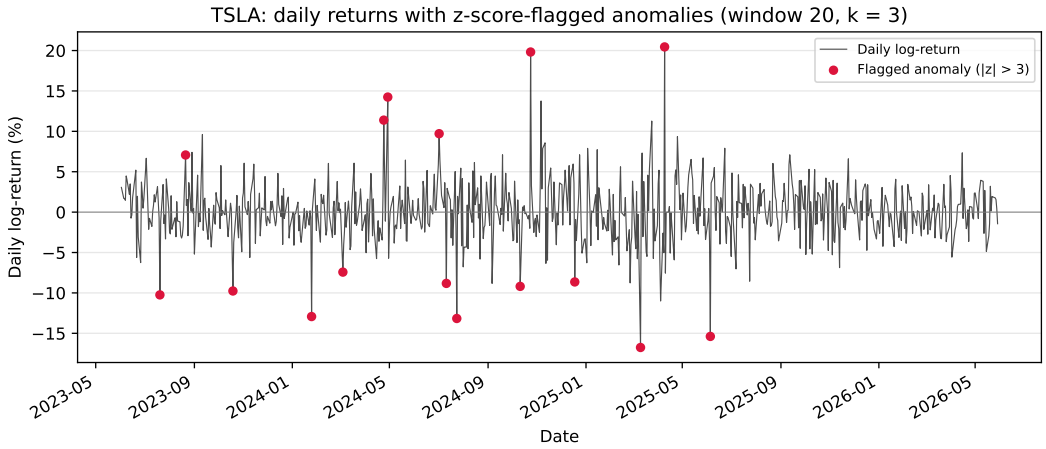

Figure 5.2: Daily log-returns of a volatile stock (Tesla) over the evaluation window, with the days flagged by the z-score detector (window 20, $k = 3$) marked. The detector responds to volatility-relative surprise rather than to absolute move size.

the move, the recent norm it is judged against, and the resulting z-score, which is what the explanation engine then shows the investor.

Table 5.2: Worked anomaly: Tesla's post-earnings move on 24 October 2024 (window 20, $k = 3$); real, reproducible figures.

Quantity	**Value**
Trailing-20-day mean of log-returns, μ	-0.92%
Trailing-20-day standard deviation, σ	2.73%
Day's log-return, r (a price move of $\approx +22\%$)	$+19.82\%$
z-score, $(r - \mu)/\sigma$	$\mathbf{+7.61}$
Flagged at $k = 3$?	yes

Agreement with the proxy label (supporting). Against the per-ticker extreme-move label, the z-score attains an F_1 of 0.516 (precision 0.381, recall 0.800) versus 0.218 for the fixed threshold (precision 0.122, recall 0.992): the fixed rule catches almost every large raw move but at very low precision. A window ablation (Figure 5.3) shows F_1 rising with the estimation window (0.385, 0.516 and 0.678 for 10, 20 and 60 days), indicating that a longer volatility estimate yields a steadier baseline before it begins to saturate. This label is itself volatility-relative, so some circularity is unavoidable; that is why the firing-rate consistency above, which depends on no label, is treated as the primary evidence.

Statistical rule versus learned detector. The protocol of Section 3.6.3 closes with a challenge: would a *learned* detector, given exactly the same information, beat the transparent rule? An Isolation Forest (F. T. Liu, Ting, and Zhou 2008) was trained causally per ticker on each day's return and the volatility of the twenty preceding days, fitted on an initial 250-day segment only, and both detectors were scored on the identical remaining region. The learned model loses clearly: against the extreme-move proxy it reaches an F_1 of 0.271 (precision 0.159, recall 0.913), while the z-score on the same region reaches 0.530 (precision 0.407,

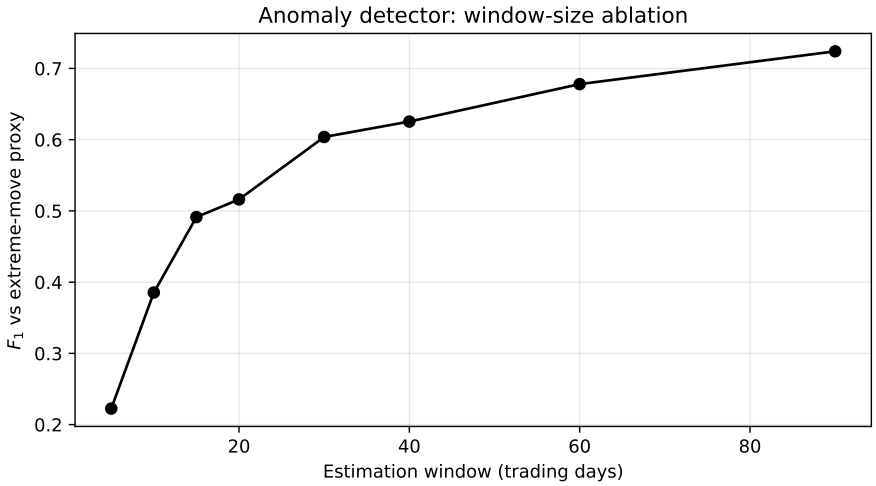

Figure 5.3: Effect of the estimation-window length on the detector's F_1 against the extreme-move proxy. A longer window yields a steadier volatility baseline and higher agreement, with diminishing returns at the longest windows.

recall 0.761); the forest also fires at rates that vary across tickers by 0.135, two orders of magnitude less uniform than the z-score's 0.015, failing the consistency requirement that motivates the design. The comparison, rather than any assumption, is what justifies keeping the statistical rule in production: given identical causal information, the interpretable detector was not merely simpler but better.

Extension: a second learned detector, and a second volatility estimate. Two natural objections remain: perhaps a *different* learned detector would win, and perhaps a more adaptive volatility estimate would. Both were tested under the identical causal protocol (Section 3.6.3), in a fresh rerun whose z-score row reproduces the frozen values above exactly (the forest shifts in the third decimal because the free price source re-adjusts historical closes retroactively). Table 5.3 and Figure 5.4 report both answers. The Local Outlier Factor (Breunig et al. 2000), the density-based alternative of Chapter 2, behaves like the forest: it recalls almost everything (0.989) at very low precision (0.163), lands at an F_1 of 0.280, and fires at rates that vary across tickers by 0.183, the least consistent of the three. The transparent rule beats both learned detectors on the same information.

The volatility question produced the more interesting answer, and it is reported exactly as it fell. Replacing the rolling standard deviation with an exponentially weighted estimate (RiskMetrics, $\lambda = 0.94$), the first step toward the GARCH family, *improves* agreement with the extreme-move proxy substantially: at the same recall (0.800), precision rises from 0.381 to 0.568, lifting F_1 from 0.516 to 0.664, with per-ticker firing rates slightly *more* uniform (0.012 versus 0.015). The mechanism is volatility clustering itself: after a shock, a rolling window dilutes the new regime across twenty equal weights, whereas the exponential estimate absorbs it at once and stops re-flagging ordinary days that follow. The experiment was designed to license the phrase "GARCH would add little", and it does not license it; what it shows instead is that the first adaptive step already helps *on this proxy*. The deployed detector keeps the rolling rule, because "the average and spread of the last twenty days" is explainable to a non-expert in one sentence while exponential weights are not, but the gain

is recorded as validated rather than speculative future work, one line of configuration away. The proxy's own volatility-relative circularity, assumed throughout this case study, applies here equally.

Table 5.3: Extended comparison under the frozen causal protocol. Top: detectors on the identical scored region and features. Bottom: the same z-score rule with two volatility estimates, over the full evaluation region. Reproducible via `scripts/evaluate_anomaly_ext.py`.

Detector (same region)	**Precision**	**Recall**	F_1	**Rate range**
z-score (deployed rule)	0.407	0.761	**0.530**	**0.017**
Isolation Forest	0.158	0.913	0.269	0.135
Local Outlier Factor	0.163	0.989	0.280	0.183
Volatility estimate (z-score)				
Rolling σ, 20 d (deployed)	0.381	0.800	0.516	0.015
EWMA σ ($\lambda = 0.94$)	0.568	0.800	**0.664**	**0.012**

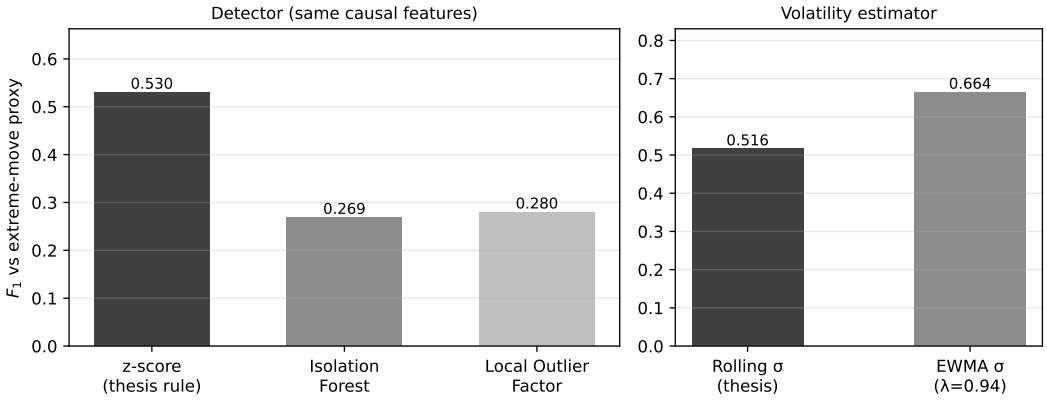

Figure 5.4: The extended comparison at a glance. Left: on identical causal features, the transparent z-score rule beats both learned detectors. Right: the exponentially weighted volatility estimate outperforms the rolling one on the proxy, a finding reported as it fell; the rolling rule stays deployed for explainability, with the EWMA gain recorded as validated future work.

5.3 Case Study 2: News–Market Precedent Retrieval

Does semantic retrieval find genuinely analogous precedents, better than trivial baselines? **Yes**: on real data the embeddings beat every baseline by a wide margin.

The metric is precision@k with a sector proxy, in a *cross-ticker* setting: for each query the k most similar headlines *from other companies* are retrieved, and precision is the share that match the query's sector. Excluding the query's own company rules out the trivial win of matching a company to its own news, and tests thematic analogy instead. Same-sector is an automatic stand-in for "analogous", so the metric shows whether retrieved news is on-theme, not whether a person would call it useful; that gap is real, and a human study is future work. To show the result is not tied to one model, two Sentence-BERT models (Reimers and Gurevych 2019) (the lightweight MiniLM and the larger MPNet) are compared

against a lexical (hashing) baseline and two trivial ones, random (the sector base rate) and recency. Five hundred queries were sampled in each of five runs (seeds 42–46); Table 5.4 reports the mean and standard deviation.

Table 5.4: Cross-ticker precision@k by sector, mean $\pm$ std over five runs (3,714 real headlines, five sectors).

Method	**P@5**	**P@10**
SBERT (MiniLM, default)	0.514 ± 0.015	0.478 ± 0.011
SBERT (MPNet)	$\mathbf{0.538 \pm 0.011}$	$\mathbf{0.506 \pm 0.010}$
Lexical baseline (hashing)	0.346 ± 0.011	0.314 ± 0.008
Recency	0.126 ± 0.006	0.106 ± 0.005
Random (base rate)	0.240 ± 0.004	0.240 ± 0.004

As Table 5.4 and Figure 5.5 show, the default MiniLM model reaches a precision@5 of 0.514, about 2.1 times the random base rate (0.240) and well above the lexical baseline (0.346). The stronger MPNet model does slightly better (0.538), which suggests the advantage belongs to semantic embeddings in general rather than to one particular model; the small standard deviations (around 0.01) show the gap is robust rather than a sampling artefact. Recency performs worst, below even the random base rate, so simply surfacing the latest news is no substitute for semantic matching. The lift over both the random and lexical baselines is the honest measure of the value added by the embeddings.

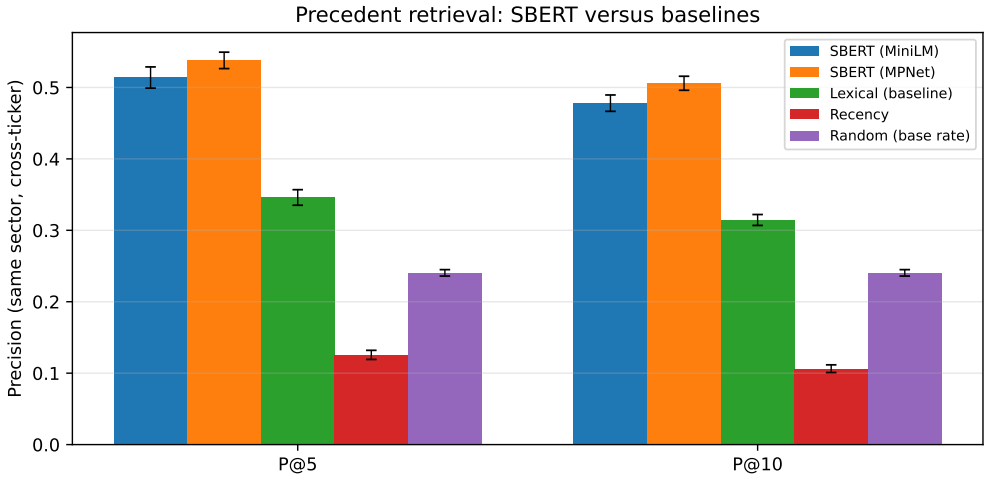

Figure 5.5: Cross-ticker precision@k by sector. SBERT clearly exceeds the lexical, recency and random baselines; error bars show ± 1 standard deviation over the five runs.

Seeing the space. The embedding space itself, so far only sketched conceptually (Figure 3.4), can be shown for real. Figure 5.6 is a principal-component projection of the production knowledge base, all 2,016 cases with their genuine 384-dimensional MiniLM embeddings, coloured by sector, with a live query (the worked example of Chapter 3) marked as a star and its three retrieved neighbours circled. Two honest readings follow. Even this two-dimensional shadow, which preserves only 8.6% of the variance, places the query inside its topical neighbourhood, and the three retrieved cases (all semiconductor-demand stories,

at cosines 0.58–0.61 in the full space) sit visibly adjacent to it. And the sectors do *not* separate cleanly in two dimensions, which is exactly why the system ranks by cosine in the full 384-dimensional space rather than in any projection: the picture is for intuition, the ranking is not done there.

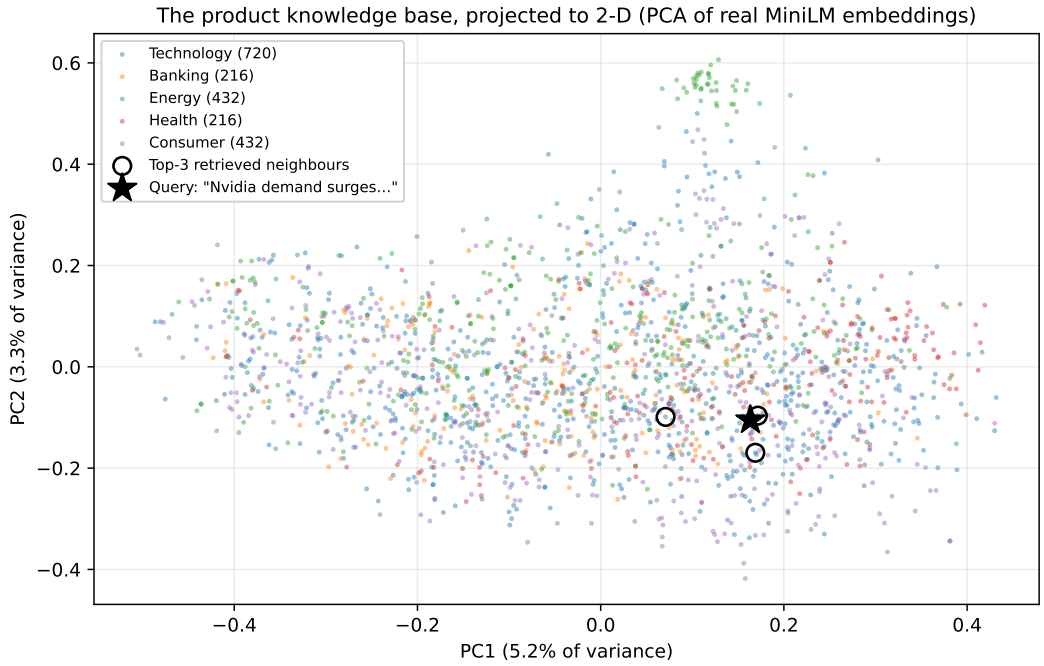

Figure 5.6: The production knowledge base projected to two dimensions (PCA over the real MiniLM embeddings; axes show the variance each component preserves). Colours are sectors; the star is the query *"Nvidia demand surges on AI chip orders"*; circles mark its top-3 retrieved neighbours. The projection is a shadow for intuition: retrieval ranks by cosine in the full 384-dimensional space. Reproducible via `scripts/figures/fig_embedding_projection.py`.

Impact measurement. For each retrieved precedent the engine reports the observed post-event return at +1, +3 and +5 trading days, measured from the event-day close in the style of an event study (Brown and Warner 1985). These figures are observed outcomes presented as evidence, never a prediction. On the worked knowledge base the measured impacts are directionally coherent (for example, the competitive-threat news cluster in the end-to-end case study below is followed by uniformly negative one-day returns), which provides face validity for the measurement; a rigorous multi-year impact study on the full FNSPID corpus is identified as future work.

Retrieval by sector. The aggregate figure hides useful variation across sectors. Table 5.5 and Figure 5.7 break the retrieval down by the query's sector, using the whole population of each sector as queries (so the figures are exact rather than sampled). Two readings matter. The raw precision@5 is highest for technology (0.712), but largely because technology dominates the corpus (47% of headlines) and therefore has a high random base rate (0.429); the fair cross-sector measure is the *lift* over that base rate. By that measure the engine adds the most value in **energy** (+0.377) and **health** (+0.348), sectors whose vocabulary

is distinctive (production, output, barrels; trial, drug, approval), and the least in **consumer** (+0.100), whose headlines are more generic and thematically overlap with other sectors. Retrieval is positive in every sector, but its benefit is greatest where the domain language is most specific.

Table 5.5: Cross-ticker precision@k per sector (whole-population queries, MiniLM).

Sector	N	P@5	P@10	Random	Lift P@5
Technology	1736	0.712	0.675	0.429	+0.283
Energy	497	0.448	0.408	0.072	**+0.377**
Health	494	0.419	0.379	0.071	+0.348
Banking	496	0.272	0.228	0.072	+0.200
Consumer	491	0.171	0.150	0.071	+0.100

Note: the lift is precision@5 minus the random base rate, computed from unrounded values; it may differ from the rounded columns by ±0.001.

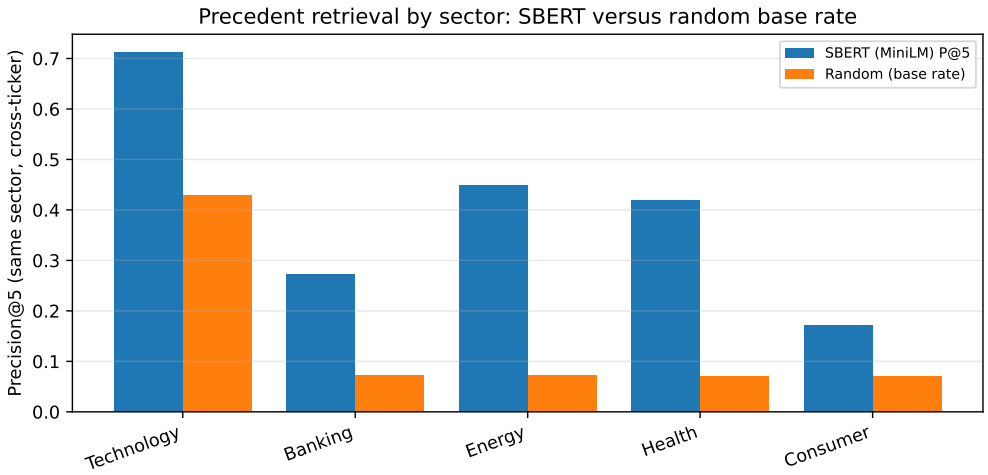

Figure 5.7: Precision@5 per sector against the random base rate. Retrieval exceeds the base rate in every sector; the gap (lift) is largest for energy and health, whose vocabulary is most distinctive, and smallest for the broad consumer sector.

Why some sectors retrieve better: a qualitative look. The per-sector gap has a concrete explanation, visible in individual queries. In technology, different companies genuinely share themes: a headline about NVIDIA's data-centre accelerators retrieves, from the Microsoft, Meta and Alphabet feeds, articles about the same wave of AI-chip competition (Case Study 3), so cross-ticker precision is high. The consumer sector behaves differently. Looking at the raw neighbourhood (before the cross-ticker constraint is applied), a query about Walmart's grocery guidance retrieves mostly Walmart's own items plus a macro "US retail sales" story carried on bank feeds: adjacent in theme, but not a consumer analogue from another consumer company. Once the query's own ticker is excluded, as the metric requires, little genuinely relevant material is left, which is why the cross-ticker lift for consumer is low. A query about Coca-Cola retrieves almost only other Coca-Cola items, about its pricing power and beverage volumes, which simply do not resemble Walmart's store-traffic news.

The shared label "consumer" is too broad: its members' news is idiosyncratic, so cross-ticker analogues are scarce and the lift over the base rate is small. This is the pattern the numbers show, and it points to a clear refinement for future work: grouping by finer-grained themes rather than by sector.

5.4 Case Study 3: End-to-End Alert and Explanation Faithfulness

Does a complete alert hold together, and is its explanation faithful? **Yes, faithful by construction.** To see the news trigger end to end, take a new NVIDIA headline, *"Nvidia raises guidance on surging demand for AI data-centre accelerators"*. The system embeds it, retrieves the most similar precedents from the knowledge base built on the real news corpus, and renders the alert below (top five precedents, one-day horizon):

```
News alert for NVDA
"Nvidia raises guidance on surging demand for AI data-centre
accelerators"
Potential impact (from 5 similar past events):  average 1-day move:
−1.97%
Historical precedents:
- 2026-06-24 MSFT (sim 0.68) +1d −3.46%:  "Qualcomm debuts line of AI
data center chips..."
- 2026-06-24 NVDA (sim 0.68) +1d −1.64%:  "Qualcomm debuts line of AI
data center chips..."
- 2026-06-24 META (sim 0.68) +1d −2.65%:  "Qualcomm debuts line of AI
data center chips..."
- 2026-06-24 GOOGL (sim 0.64) +1d −0.46%:  "This AI Infrastructure
Stock Could Be Bigger Than Nvidia..."
- 2026-06-24 NVDA (sim 0.58) +1d −1.64%:  "Tecan Accelerates
Data-Driven Lab Journey...  NVIDIA"
Note:  precedents are retrieved by semantic similarity; the impact is
the observed past outcome, not a price prediction.
```

Every retrieved precedent concerns AI data-centre chips and competition for NVIDIA, even though the articles were filed under different company feeds (Microsoft, Meta, Alphabet and NVIDIA itself). Unlike the retrieval metric of Section 5.3, the live alert does not exclude the query's own ticker: that exclusion exists only to keep the evaluation non-trivial, whereas a real user is well served by the most similar precedents, whichever company they concern. Strikingly, the same story (a rival entering the AI data-centre chip market) surfaces across three different company feeds, which the engine correctly groups as one theme. This is direct evidence that retrieval is driven by *meaning* rather than the company name or exact keywords, the very behaviour the correlation engine is designed for. In this instance the precedents happen to be uniformly negative (a competitive-threat cluster), and their average is reported as an observed past outcome, not a prediction. The alert ends with an explicit disclaimer, keeping the system within its no-forecasting scope.

What the retrieval does and does not capture. This example also exposes an honest limitation. The query headline reads as positive (guidance raised), yet the precedents it retrieves form a competitive-threat cluster whose one-day outcomes are uniformly negative. There is no contradiction once the mechanism is clear: sentence-embedding similarity

matches on *theme*, here "AI data-centre chips", and not on sentiment or expected direction. The mean impact the alert reports is therefore evidence about how a theme has tended to move, not a directional forecast for this particular item, which is exactly why the alert always lists the individual precedents and closes with a no-prediction disclaimer rather than leaning on a single averaged number. Treating that average as a prediction would be the over-reliance that the trust literature warns against (Bansal et al. 2021; J. D. Lee and See 2004), and the design guards against it by showing the evidence rather than issuing a verdict.

Two cosmetic artefacts of the shallow recent corpus are also visible and worth naming plainly: the precedents share a single retrieval date, because the free news tier returns only a recent window and there is little genuine history to draw on, and the same ticker can appear twice with an identical impact, because same-ticker precedents aligned to the same event day necessarily share their forward return. Both follow from corpus depth, not from the method, and both disappear once the full multi-year knowledge base is built. For the same reason, the positive mean impact obtained for a similar Nvidia query on the bundled sample base in Section 3.3.2 (+6.5% at five days) and the negative mean here (−1.97% at one day) are not in tension: they use different knowledge bases, horizons and encoders, and neither is a forecast.

Explanation faithfulness. Faithfulness is guaranteed by construction: the alert text is rendered directly from the same objects the system computes (the z-score, window and threshold for the anomaly path, and the exact retrieved precedents with their similarity scores and measured impacts for the news path), so the explanation cannot diverge from the underlying computation. This is checked programmatically by an automated test that asserts the rendered alert reproduces exactly each retrieved precedent's date, ticker and similarity score and introduces none that were not retrieved; it is the kind of transparent, design-time explainability advocated in the XAI literature (Barredo Arrieta et al. 2020). Usefulness to a retail investor is inherently subjective; a small rubric-based human assessment (clarity, completeness, actionability) is planned but has not yet been conducted, and is therefore reported as a limitation rather than a result.

5.5 Case Study 4: Learned Materiality Triage

Question: can the trained triage model of Section 3.3.3 prioritise news better than simple volatility evidence (RQ4)? Answer: every trained ranker clears the always-alert floor by a wide margin, and within a realistic alert budget triage nearly quadruples precision; but none of the models that read the headline text beats the volatility-only baseline. On this evidence, the triage signal lives in market context, not in headline wording, and the dissertation reports that finding as it stands.

Corpus and setup. The FNSPID subset was streamed and aligned to prices exactly as designed: 79,753 examples over 1,501 unique trading days (January 2018 to December 2023), with zero discarded rows. Fourteen of the fifteen tickers are represented; Meta is absent because the corpus indexes it under its former symbol (FB) for most of the window, a coverage note rather than a method change. The temporal split by unique days (70/15/15, five-day embargo) yields 28,574 training, 17,710 validation and 32,649 test rows, with material examples at 38.5%, 47.0% and 37.8% respectively. Two corpus properties matter for reading the results. News density grows over time (2023 alone holds 44% of the examples), so the later blocks hold more rows than their share of days. And headlines from the same

(ticker, day) pair share one label, so rows come in clusters; the split by unique days keeps each cluster inside a single block.

Results. Table 5.6 reports the test block; Figure 5.8 shows the precision–recall curves and Figure 5.9 the calibration of the main interpretable model.

Table 5.6: Materiality triage on the FNSPID corpus (test block; prevalence 0.378). PR-AUC is the primary metric; precision@budget admits at most five alerts per day.

Model	PR-AUC	ROC-AUC	Brier	Prec.@budget
Always alert (floor)	0.378	0.500	0.622	0.163
Volatility-only LR (baseline)	**0.542**	**0.665**	**0.218**	**0.632**
Context-only LR	0.538	0.658	0.224	**0.632**
Text-only LR	0.439	0.564	0.240	0.572
Context+text LR (main)	0.496	0.622	0.229	0.585
Gradient boosting (context+text)	0.469	0.624	0.228	0.551

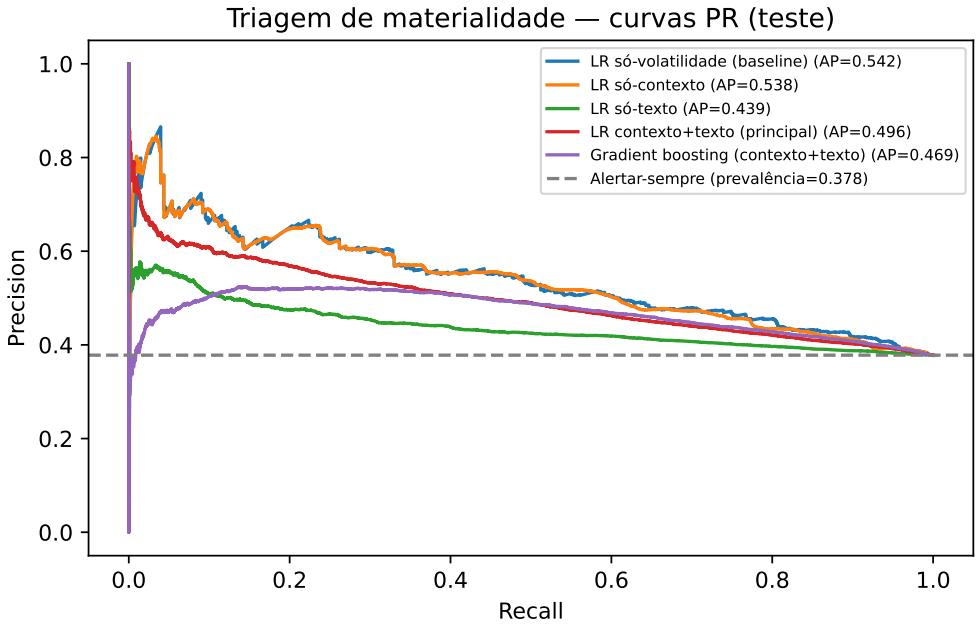

Figure 5.8: Precision–recall curves on the test block. All trained models sit well above the always-alert floor (dashed line at the prevalence); the volatility-only baseline is not overtaken by any model that uses the headline text.

Reading the result. Three observations, in decreasing order of practical weight. First, *triage works as a product mechanism*: within a budget of five alerts per day, ranking by any trained score raises the share of material alerts from 0.163 (picking blindly) to 0.632, nearly four times better, with calibrated probabilities (Brier 0.218 against 0.622 for the floor). This is the alert-fatigue payoff the component was built for. Second, *the signal is market context, not text*: the volatility-only baseline attains the best PR-AUC (0.542), the context-only model matches it in budgeted precision, and adding the headline embedding

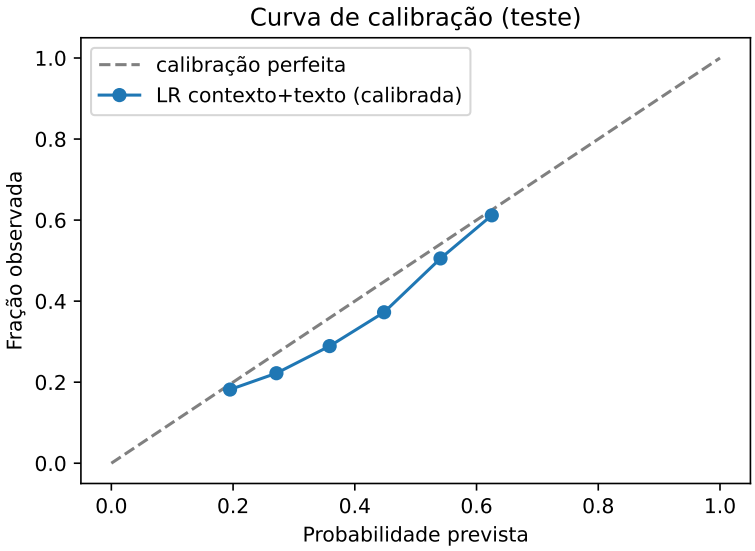

Figure 5.9: Calibration of the main interpretable model after Platt scaling on the validation block: predicted probabilities track observed frequencies, which is what makes the score honest to show to an end user.

lowers the linear model to 0.496; the gradient-boosted ensemble does not recover the gap (0.469). Headlines alone are far above the floor (0.439) but well below volatility. Third, the answer to RQ4 as posed is therefore *negative on the text hypothesis and positive on the mechanism*: a supervised, calibrated ranker adds real product value over naive alerting, but on this corpus and with this text representation it does not beat the simple volatility baseline it was required to beat. This is the second time in this chapter that a learned method was given a fair, causal comparison against the transparent choice and lost (Case Study 1); both comparisons were pre-committed in Section 3.6, and both validate the dissertation's simplicity-first design with evidence rather than assumption. The deployed variant (context-only, Section 4.5) is consistent with this finding: it uses exactly the features that carry the signal.

Caveats. The label is a proxy: market-adjusted move size, not human judgement of materiality. Same-day headlines share one label, so the effective sample size is smaller than the row count, and the corpus's news density grows over the window. Headlines are short, so a richer text representation (article bodies, financial sentiment) might yet close the text gap; that possibility is recorded as future work rather than assumed.

The deployed model on one real decision. The mechanism claim becomes tangible on a single live case. Figure 5.10 decomposes a real production decision, the Meta alert of 12 July 2026 whose full arithmetic is Table 3.5: elevated recent volatility and the sector indicator carry essentially the whole log-odds, the text-free near-zero terms confirm that context is the signal, and the calibrated 54% that reached the public channel is reproduced exactly by the decomposition. The evaluation's aggregate finding (context, not text) and the deployed product's individual decisions are one and the same computation, which is the faithfulness property the design promises.

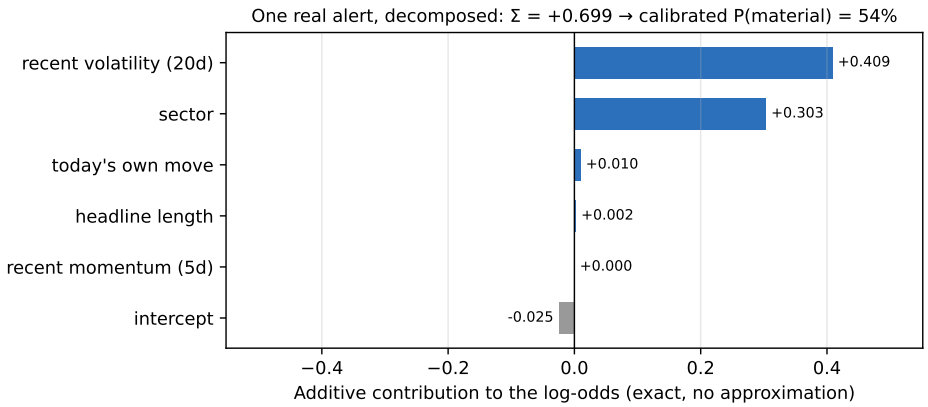

Figure 5.10: One real production triage decision (META, 12 July 2026), decomposed into its exact additive log-odds contributions. Recent volatility and sector dominate; the sum +0.699 maps through the sigmoid and the Platt calibration to the 54% actually sent to the channel (Table 3.5). Reproducible via `scripts/figures/fig_triage_worked.py`.

Do more market features help? A feature ablation. The result above says the signal lives in market context rather than headline text, which invites the natural follow-up: would *more* context help? Five additional signals were engineered, all cheap (no new data source) and all under the same causal convention as the originals, so that nothing after the event day can leak into them: the broad market's own volatility regime (on the market index), the stock's twenty-day momentum, the ratio of short to long volatility, the standardised immediate reaction (the detector's own z), and a downside-only volatility. The context-only model was retrained on these under the frozen protocol (temporal split, Platt calibration on validation, seed 42). The volatility anchor reproduces the frozen number (0.537 against the frozen 0.538, the tiny gap coming only from the stricter sixty-day history the extended features require). Adding all five signals moves PR-AUC to 0.535 — no improvement. A leave-one-in / leave-one-out sweep (Figure 5.11) explains why: the standardised reaction is the only signal with a positive sign, and only by 0.001; the others are flat or slightly negative. Rolling volatility already absorbs almost everything these cheap features carry, which is the same lesson as the text result, reached from the opposite direction: on this corpus, near-term materiality is remarkably well summarised by one number. The finding is reported exactly as it fell and changes nothing in production; the full table is in `docs/evaluation/evaluation_triage_ext.md`, reproducible via `scripts/train_triage_ext.py`.

5.6 Discussion and Threats to Validity

The four case studies support the core design: volatility-normalised detection is consistent across the market where a fixed threshold is not, semantic retrieval surfaces markedly more sector-analogous precedents than lexical or trivial baselines, the end-to-end alert is faithful to its own computation, and the learned triage adds a large budgeted-precision gain while confirming, against its own text features, that the transparent volatility evidence carries the signal. All results rest on real data and are reproducible from versioned scripts.

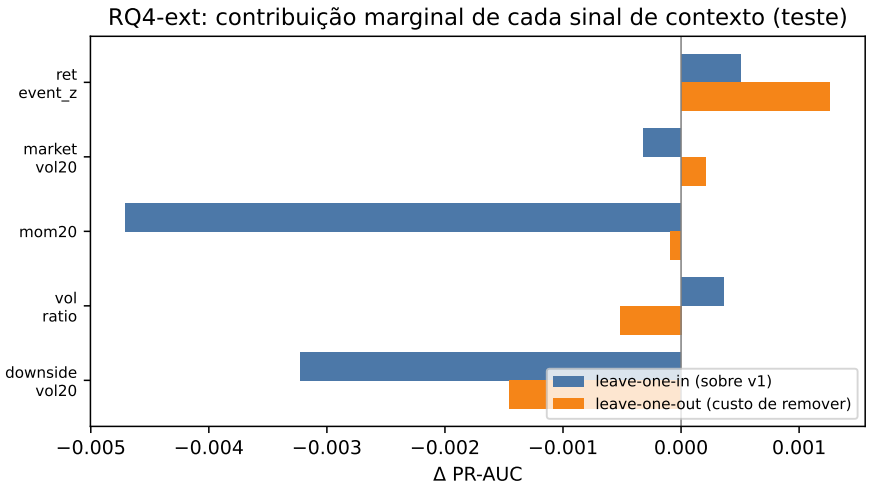

Figure 5.11: Marginal PR-AUC contribution of each extended context feature on the test block, measured two ways: added on its own to the volatility model (leave-one-in) and removed from the full set (leave-one-out). All effects are within ±0.005 of zero: the cheap engineered signals are redundant given rolling volatility. Reproducible via `scripts/train_triage_ext.py`.

The results also have clear limits, which are set out below under the four standard categories of validity so that nothing is glossed over.

Construct validity: do the metrics measure what is claimed? Two proxies carry weight here. Retrieval relevance is judged by a *sector* label, an automatic stand-in for the human judgement of "analogous", so a same-sector precedent on an unrelated topic counts as a hit; the qualitative inspection in Section 5.3 shows this proxy is reasonable but imperfect. Likewise, the anomaly "extreme move" label is volatility-relative, so it is not an independent ground truth, which is why the primary anomaly argument is the label-free *firing-rate consistency*, not agreement with that label. Finally, the event-study figure measures the price change that *followed* a news item, which is an outcome, not a causal effect of the news; it is reported as evidence, never as a forecast.

Internal validity: could something else explain the effect? The retrieval advantage might, in principle, be inflated by trivially matching a company's news to its own past news; this is removed by construction, because the evaluation forbids same-ticker precedents and measures *cross-ticker* precision only. Lookahead is excluded throughout: every quantity uses information available at the time, and impacts are accumulated strictly forward from the event-day close. The main residual threat is confounding (a price move in a post-event window may be driven by market-wide factors rather than the news), which the short windows (+1, +3, +5 days) limit but do not eliminate, and which the no-causation framing acknowledges openly.

External validity: how far do the findings generalise? The study covers fifteen large-capitalisation US tickers across five sectors over a recent window, so the numbers need not transfer to small-capitalisation stocks, to other markets, or to other periods, and news coverage is thinner outside large caps. The corpus is a recent window from a free service

rather than the multi-year FNSPID history; the full corpus is identified as the natural way to broaden this scope.

Statistical-conclusion validity: are the comparisons sound? The retrieval results are averaged over five random samples with standard deviations reported (the error bars in Figure 5.5), and the gap between semantic retrieval and every baseline is large relative to that spread; still, five runs on one corpus is a modest sample, and no formal significance test is claimed. The anomaly comparison is reported descriptively, as a difference in firing rates across instruments, rather than as a hypothesis test.

One limit is deliberate rather than a weakness: the system makes no price prediction and evaluates no trading profit, so those questions are out of scope by design. The full FNSPID corpus, a human relevance and usefulness study, and a multi-year impact analysis are the natural next steps, and are carried forward to the conclusions.

5.7 Chapter Conclusions

Across four case studies on real data, InvestiGator's anomaly trigger fired far more consistently than a fixed threshold, its correlation engine retrieved sector-analogous precedents well above every baseline and independently of the embedding model, its explanations were faithful to the underlying computation by construction, and its trained triage model nearly quadrupled within-budget alert precision while honestly failing to beat the volatility-only baseline with text. The next chapter draws these findings together against the research questions.

Chapter 6

Conclusions

This dissertation set out to design, implement and evaluate an explainable, reproducible financial-alert system for US retail investors, driven by two independent triggers and delivered through Telegram. This chapter summarises what was achieved against the research questions, revisits the contributions, and states the limitations and the directions they open.

6.1 Main Conclusions

The work is best summarised by returning to the four research questions of Chapter 1.

RQ1: transparent detection of abrupt movements. Yes. A rolling z-score detector flags abnormal returns and, more importantly, does so at a steady rate *across stocks of very different volatility*, where a fixed-percentage threshold cannot. As reported in Chapter 5, the firing rate varied by only 0.015 across the fifteen tickers for the z-score versus 0.344 for the fixed baseline, and the z-score reached an F_1 of 0.516 against an extreme-move proxy, more than double the baseline. The method is simple enough that every alert can be explained to the investor.

RQ2: analogous precedents without lookahead. Yes, for retrieval. Semantic retrieval with Sentence-BERT recovered markedly more sector-analogous precedents than every trivial alternative: a cross-ticker precision@5 of 0.514 ± 0.015 (over five runs), against 0.346 for a lexical baseline, 0.240 for random and 0.126 for recency. A per-sector breakdown showed the lift is greatest where domain vocabulary is most distinctive (energy, health). The end-to-end case study showed it surfacing genuinely topical precedents across different company feeds. Impact is measured strictly after the event, in the style of an event study, so no future information leaks into the evidence. The measurement machinery is in place and behaves coherently; a large-scale, multi-year validation of impact magnitudes remains for future work.

RQ3: faithful and useful explanations. Faithful, yes; useful, still open. Because each alert is rendered directly from the same objects the system computes (the z-score and its parameters, or the retrieved precedents with their scores and measured impacts), the explanation cannot diverge from the underlying logic, and every alert closes with an explicit no-prediction disclaimer. Usefulness to a retail investor is plausible by design but has not yet been measured with a human study, and is therefore reported as an open point rather than a settled result.

RQ4: learned triage beyond volatility baselines. No on the text hypothesis; yes on the mechanism. A supervised materiality model was trained, calibrated and tested on 79,753 FNSPID examples (2018–2023) under a strict temporal protocol. As a product mechanism, learned triage is clearly worthwhile: within a five-alerts-per-day budget it raised the share of material alerts from 0.163 to 0.632, with calibrated probabilities. But the pre-committed decisive comparison went the other way: no model reading the headline text beat the volatility-only baseline (PR-AUC 0.542 against 0.496 for the main context+text model), so on this corpus the triage signal lives in market context, not headline wording. Together with the Isolation Forest comparison of the anomaly study, this is the second fair, causal test in which the transparent choice won, and both results are reported exactly as they fell.

6.2 Research Questions and Objectives Achieved

The general objective, to design, implement and evaluate an explainable, reproducible alerting system driven by two triggers, was met: InvestiGator is a working system whose two triggers were proven end to end and whose core components, including the trained triage model, were evaluated on real data. The supporting objectives were also met: a thin end-to-end slice was delivered first; each component was kept in its simplest defensible form; the evaluation followed a design fixed in advance; and the whole system is reproducible from versioned scripts.

6.3 Contributions Revisited

As an Artificial Intelligence *Engineering* dissertation, the contribution is the integration, application and critical evaluation of existing components into a working, explainable and reproducible whole, rather than a new algorithm. Four concrete contributions stand out. First, a documented and reproducible methodology for **news–market impact correlation** that combines sentence-embedding retrieval with event-study impact windows, with explicit choices of similarity metric and horizon and explicit avoidance of lookahead. Second, **InvestiGator**, an **XAI-first alerting system** whose entire reasoning chain (detection, evidence, sources and precedents) is exposed to a non-expert user and is faithful by construction. Third, a trained and calibrated **materiality-triage model**, labelled by the system's own event-study code over the multi-year corpus, integrated into the alerts off by default as ranked and explained evidence, and continually post-validated in deployment against the outcomes that actually followed. Fourth, a **critical, honest evaluation** of each core component against trivial, lexical and volatility baselines on real data, including two pre-committed statistical-versus-learned comparisons that the transparent methods won; results, ablations and limitations are all reported plainly and regenerated from versioned scripts.

6.4 Limitations

Several limitations remain, stated plainly:

- **Corpus.** The retrieval evaluation used a recent, few-month news corpus from a free service, which bounds the impact analysis and makes the retrieval figures, though real, preliminary. The triage study did use the multi-year FNSPID subset, but with fourteen of the fifteen tickers (Meta is indexed under its former symbol in the corpus) and headline text only.

- **Proxies.** Relevance is proxied by sector agreement, and the anomaly label is volatility-relative, which is why the label-free firing-rate argument is given primacy.
- **Short text.** News is represented by headlines only, which limits the semantics captured.
- **Theme, not direction.** Embedding similarity captures theme, not direction, so a retrieved cluster can mix up and down moves; the mean impact is theme-level evidence, not a directional signal, which is why the precedents are always shown individually.
- **No human study.** Usefulness to a real investor has not yet been measured.
- **By design.** The system makes no price prediction and no trade, so profit-based questions are out of scope, not unanswered.

6.5 Future Work

The limitations map directly onto future work:

- Evaluate *retrieval* on the multi-year FNSPID corpus. The knowledge base itself has already been rebuilt from that corpus (after the case studies here were frozen), so the open work is the evaluation: a richer impact analysis with dispersion and direction-consistency statistics over the multi-year precedents.
- Run a small human study, applying a clarity, completeness and actionability rubric to a set of alerts, to settle the open usefulness question (RQ3).
- Attack the open text gap of RQ4: richer text than headlines (article bodies, financial sentiment as a feature), the missing ticker recovered under its former symbol, and the live post-validation loop accumulating fresh labelled decisions for periodic retraining; a contextual-bandit treatment of the alert threshold is the natural learned extension.
- As a product, de-duplicate near-identical precedents and flag directional disagreement within a retrieved cluster, complementing the severity ranking that triage already provides against the alert-fatigue and over-reliance risks of Chapter 4.

Live deployment after the evaluation was frozen already motivated a first iteration of several of these items: the deployed system now grows a *live* knowledge base from the headlines it scans (each case matured against observed prices days later), ranks precedents with an age-decay factor while always displaying the true similarity and the age of each case, flags directional disagreement explicitly, and evaluates the day's move in progress from a real-time quote during market hours. Deployment also taught its own lesson, reported in Chapter 4: a single free price source proved a single point of failure on hosted infrastructure, and was replaced by a multi-provider fallback chain. These deployment-side iterations reuse the validated components unchanged and are reported here as engineering follow-through; their formal evaluation remains future work. One item is already *validated* rather than speculative: the extended comparison of Case Study 1 showed that an exponentially weighted volatility estimate improves the detector's proxy agreement substantially, so adopting it is a measured, one-line change waiting only on an explainability decision. Together these would turn a sound prototype into a more thoroughly validated system, without giving up the simplicity and transparency that make it defensible.

Bibliography

Aamodt, Agnar and Enric Plaza (1994). "Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches". In: *AI Communications* 7.1, pp. 39–59. doi: 10.3233/AIC-1994-7104.

Adadi, Amina and Mohammed Berrada (2018). "Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI)". In: *IEEE Access* 6, pp. 52138–52160. doi: 10.1109/ACCESS.2018.2870052.

Ahmed, Mohiuddin, Abdun Naser Mahmood, and Md. Rafiqul Islam (2016). "A Survey of Anomaly Detection Techniques in Financial Domain". In: *Future Generation Computer Systems* 55, pp. 278–288. doi: 10.1016/j.future.2015.01.001.

Araci, Dogu (2019). "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models". In: *arXiv preprint*. arXiv: 1908.10063 [cs.CL].

Bansal, Gagan et al. (2021). "Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance". In: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. doi: 10.1145/3411764.3445717.

Barber, Brad M. and Terrance Odean (2008). "All That Glitters: The Effect of Attention and News on the Buying Behavior of Individual and Institutional Investors". In: *The Review of Financial Studies* 21.2, pp. 785–818. doi: 10.1093/rfs/hhm079.

Barberis, Nicholas and Richard Thaler (2003). "A Survey of Behavioral Finance". In: *Handbook of the Economics of Finance*. Ed. by George M. Constantinides, Milton Harris, and René M. Stulz. Vol. 1. Elsevier. Chap. 18, pp. 1053–1128. doi: 10.1016/S1574-0102(03)01027-6.

Barredo Arrieta, Alejandro et al. (2020). "Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI". In: *Information Fusion* 58, pp. 82–115. doi: 10.1016/j.inffus.2019.12.012.

Bollerslev, Tim (1986). "Generalized Autoregressive Conditional Heteroskedasticity". In: *Journal of Econometrics* 31.3, pp. 307–327. doi: 10.1016/0304-4076(86)90063-1.

Breunig, Markus M. et al. (2000). "LOF: Identifying Density-Based Local Outliers". In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pp. 93–104. doi: 10.1145/335191.335388.

Brown, Stephen J. and Jerold B. Warner (1985). "Using Daily Stock Returns: The Case of Event Studies". In: *Journal of Financial Economics* 14.1, pp. 3–31. doi: 10.1016/0304-405X(85)90042-X.

Cambridge Centre for Alternative Finance (2026). *Global AI in Financial Services Report: Adoption, Impact and Risks*. Cambridge Judge Business School, University of Cambridge. url: https://www.jbs.cam.ac.uk/faculty-research/centres/alternative-finance/publications/2026-global-ai-in-financial-services-report/ (visited on 06/21/2026).

Cardillo, Giovanni and Helen Chiappini (2024). "Robo-advisors: A Systematic Literature Review". In: *Finance Research Letters* 62, p. 105119. doi: 10.1016/j.frl.2024.105119.

Chandola, Varun, Arindam Banerjee, and Vipin Kumar (2009). "Anomaly Detection: A Survey". In: *ACM Computing Surveys* 41.3, 15:1–15:58. doi: 10.1145/1541880.1541882.

D'Acunto, Francesco, Nagpurnanand Prabhala, and Alberto G. Rossi (2019). "The Promises and Pitfalls of Robo-Advising". In: *The Review of Financial Studies* 32.5, pp. 1983–2020. doi: 10.1093/rfs/hhz014.

Da, Zhi, Joseph Engelberg, and Pengjie Gao (2011). "In Search of Attention". In: *The Journal of Finance* 66.5, pp. 1461–1499. doi: 10.1111/j.1540-6261.2011.01679.x.

Devlin, Jacob et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. arXiv: 1810.04805.

Ding, Xiao et al. (2015). "Deep Learning for Event-Driven Stock Prediction". In: *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2327–2333. url: https://www.ijcai.org/Abstract/15/329.

Dong, Zihan, Xinyu Fan, and Zhiyuan Peng (2024). "FNSPID: A Comprehensive Financial News Dataset in Time Series". In: *arXiv preprint*. arXiv: 2402.06698 [q-fin.ST].

Doshi-Velez, Finale and Been Kim (2017). "Towards a Rigorous Science of Interpretable Machine Learning". In: *arXiv preprint*. arXiv: 1702.08608 [stat.ML].

Engle, Robert F. (1982). "Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation". In: *Econometrica* 50.4, pp. 987–1007. doi: 10.2307/1912773.

Fama, Eugene F. (1970). "Efficient Capital Markets: A Review of Theory and Empirical Work". In: *The Journal of Finance* 25.2, pp. 383–417. doi: 10.2307/2325486.

Fama, Eugene F. et al. (1969). "The Adjustment of Stock Prices to New Information". In: *International Economic Review* 10.1, pp. 1–21. doi: 10.2307/2525569.

Friedman, Jerome H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine". In: *The Annals of Statistics* 29.5, pp. 1189–1232. doi: 10.1214/aos/1013203451.

Gallup (2025). *What Percentage of Americans Own Stock?* url: https://news.gallup.com/poll/266807/percentage-americans-owns-stock.aspx (visited on 06/21/2026).

Guidotti, Riccardo et al. (2018). "A Survey of Methods for Explaining Black Box Models". In: *ACM Computing Surveys* 51.5, 93:1–93:42. doi: 10.1145/3236009.

Johnson, Jeff, Matthijs Douze, and Hervé Jégou (2021). "Billion-Scale Similarity Search with GPUs". In: *IEEE Transactions on Big Data* 7.3, pp. 535–547. doi: 10.1109/TBDATA.2019.2921572.

Kahneman, Daniel and Amos Tversky (1979). "Prospect Theory: An Analysis of Decision under Risk". In: *Econometrica* 47.2, pp. 263–291. doi: 10.2307/1914185.

Kearney, Colm and Sha Liu (2014). "Textual Sentiment in Finance: A Survey of Methods and Models". In: *International Review of Financial Analysis* 33, pp. 171–185. doi: 10.1016/j.irfa.2014.02.006.

Lee, John D. and Katrina A. See (2004). "Trust in Automation: Designing for Appropriate Reliance". In: *Human Factors* 46.1, pp. 50–80. doi: 10.1518/hfes.46.1.50_30392.

Lipton, Zachary C. (2018). "The Mythos of Model Interpretability". In: *Communications of the ACM* 61.10, pp. 36–43. doi: 10.1145/3233231.

Liu, Fei Tony, Kai Ming Ting, and Zhi-Hua Zhou (2008). "Isolation Forest". In: *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, pp. 413–422. doi: 10.1109/ICDM.2008.17.

Loughran, Tim and Bill McDonald (2011). "When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks". In: *The Journal of Finance* 66.1, pp. 35–65. doi: 10.1111/j.1540-6261.2010.01625.x.

Lundberg, Scott M. and Su-In Lee (2017). "A Unified Approach to Interpreting Model Predictions". In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 4765–4774. arXiv: 1705.07874.

Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze (2008). *Introduction to Information Retrieval*. Cambridge, England: Cambridge University Press. isbn: 978-0-521-86571-5.

Mikolov, Tomas et al. (2013). "Efficient Estimation of Word Representations in Vector Space". In: *arXiv preprint*. arXiv: 1301.3781.

Miller, Tim (2019). "Explanation in Artificial Intelligence: Insights from the Social Sciences". In: *Artificial Intelligence* 267, pp. 1–38. doi: 10.1016/j.artint.2018.07.007.

Niculescu-Mizil, Alexandru and Rich Caruana (2005). "Predicting Good Probabilities with Supervised Learning". In: *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, pp. 625–632. doi: 10.1145/1102351.1102430.

Pang, Guansong et al. (2021). "Deep Learning for Anomaly Detection: A Review". In: *ACM Computing Surveys* 54.2, 38:1–38:38. doi: 10.1145/3439950.

Pennington, Jeffrey, Richard Socher, and Christopher D. Manning (2014). "GloVe: Global Vectors for Word Representation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532–1543. doi: 10.3115/v1/D14-1162.

Reimers, Nils and Iryna Gurevych (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin (2016). ""Why Should I Trust You?": Explaining the Predictions of Any Classifier". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144. doi: 10.1145/2939672.2939778.

Robertson, Stephen and Hugo Zaragoza (2009). "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 4.1–2, pp. 1–174. doi: 10.1561/1500000019.

Rudin, Cynthia (2019). "Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead". In: *Nature Machine Intelligence* 1.5, pp. 206–215. doi: 10.1038/s42256-019-0048-x.

Salton, Gerard, A. Wong, and C. S. Yang (1975). "A Vector Space Model for Automatic Indexing". In: *Communications of the ACM* 18.11, pp. 613–620. doi: 10.1145/361219.361220.

Securities Industry and Financial Markets Association (SIFMA) (2025). *2025 Capital Markets Fact Book*. url: https://www.sifma.org/resources/research/fact-book/ (visited on 06/21/2026).

Tetlock, Paul C. (2007). "Giving Content to Investor Sentiment: The Role of Media in the Stock Market". In: *The Journal of Finance* 62.3, pp. 1139–1168. doi: 10.1111/j.1540-6261.2007.01232.x.

Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 5998–6008. arXiv: 1706.03762.

Welch, Ivo (2022). "The Wisdom of the Robinhood Crowd". In: *The Journal of Finance* 77.3, pp. 1489–1527. doi: 10.1111/jofi.13128.

Wu, Shijie et al. (2023). "BloombergGPT: A Large Language Model for Finance". In: *arXiv preprint*. arXiv: 2303.17564 [cs.LG].

Xing, Frank Z., Erik Cambria, and Roy E. Welsch (2018). "Natural Language Based Financial Forecasting: A Survey". In: *Artificial Intelligence Review* 50.1, pp. 49–73. doi: 10.1007/s10462-017-9588-9.

Yang, Yi, Mark Christopher Siy Uy, and Allen Huang (2020). "FinBERT: A Pretrained Language Model for Financial Communications". In: *arXiv preprint*. arXiv: 2006.08097.

Appendix A

Reproducibility

This appendix records the engineering detail needed to reproduce the system and its results, kept out of the main text so that the body reads as a dissertation rather than a software specification.

A.1 Environment

The system is implemented in Python 3.12, chosen for the maturity of its packages for the machine-learning stack. Dependencies are pinned and frozen in a lock file so the environment can be recreated across machines. The numerical and data handling rely on established open-source libraries; sentence embeddings use the Sentence-BERT framework on a CPU build of the underlying deep-learning library, installed from a dedicated index to avoid the much larger GPU download; figures are produced with a standard plotting library. Table A.1 lists the pinned versions of the core dependencies; the complete set is frozen in the repository's lock file.

Table A.1: Pinned versions of the core dependencies (Python 3.12).

Library	Version
numpy	2.1.3
pandas	2.2.3
matplotlib	3.11.0
scikit-learn	1.9.0
yfinance	1.4.1
sentence-transformers	5.6.0
transformers	5.12.1
torch (CPU build)	2.12.1
pytest	8.3.4
ruff	0.8.6

A.2 Repository Organisation

The code is organised as one package per component, mirroring the architecture of Figure 4.1: market data, news retrieval, anomaly detection, the correlation engine (similarity and event study), the historical knowledge base, the explanation engine, and alert delivery, with a small orchestration layer wiring them into the two end-to-end flows. Pure logic is separated from input/output, and heavy or networked dependencies are loaded lazily, so the

numerical core is unit-tested without a network connection or the model stack. Secrets are read only from a local, ignored environment file and are never written to versioned files.

A.3 The Complete Pipeline on One Page

The body of the dissertation deliberately presents the system through several small, simple diagrams (architecture, data model, per-trigger flows, the life of one alert). Figure A.1 is the opposite view, kept in the appendix precisely because it is dense: the *complete* deployed pipeline on a single landscape page, with every gate and its production configuration value annotated. Each value is a disclosed deployment parameter (the evaluation of Chapter 5 freezes its own, stricter settings, as noted on the figure).

A.4 Tests and Reproducible Results

The system is covered by an automated test suite spanning the anomaly detector, the event study, similarity, the knowledge base, the news parser, the explanation engine and the evaluation modules, together with an offline end-to-end smoke test of the news trigger. Tests that require network access or heavy models are marked and excluded from the default run so that it is fast, deterministic and offline. All experimental results in Chapter 5 are regenerated by scripts with a fixed random seed, and the result figures are written directly into the thesis figure directory, closing the loop between code and document. The four case studies are reproduced by four commands, each writing its tables and figures into the evaluation and figure directories:

- Precedent retrieval (precision@k, Case Study 2): `python scripts/evaluate.py`
- Per-sector retrieval breakdown (Case Study 2):
  `python scripts/evaluate_per_sector.py`
- Anomaly firing-rate and window ablation (Case Study 1):
  `python scripts/evaluate_anomaly.py`
- Materiality triage: dataset build and training (Case Study 4):
  `python scripts/build_dataset.py` followed by `python scripts/train_triage.py`
- Extended detector and volatility comparison (Case Study 1):
  `python scripts/evaluate_anomaly_ext.py`
- Real embedding-space projection (Case Study 2):
  `python scripts/figures/fig_embedding_projection.py`
- Worked triage decomposition and production funnel (Chapters 3 and 4):
  `python scripts/figures/fig_triage_worked.py` and `python scripts/figures/fig_alert_funnel.`

The data-fetch step that precedes them, and the full build of the historical knowledge base, are documented in the repository's operator guide.

A.5 Proof of Work: Every Number Traced to Its Source

No result in this dissertation is typed by hand. Each script writes its numbers and figures directly into a frozen file under `docs/evaluation/`, and the thesis quotes those files, so the loop between code and document is closed and auditable. Table A.2 lists, for every headline

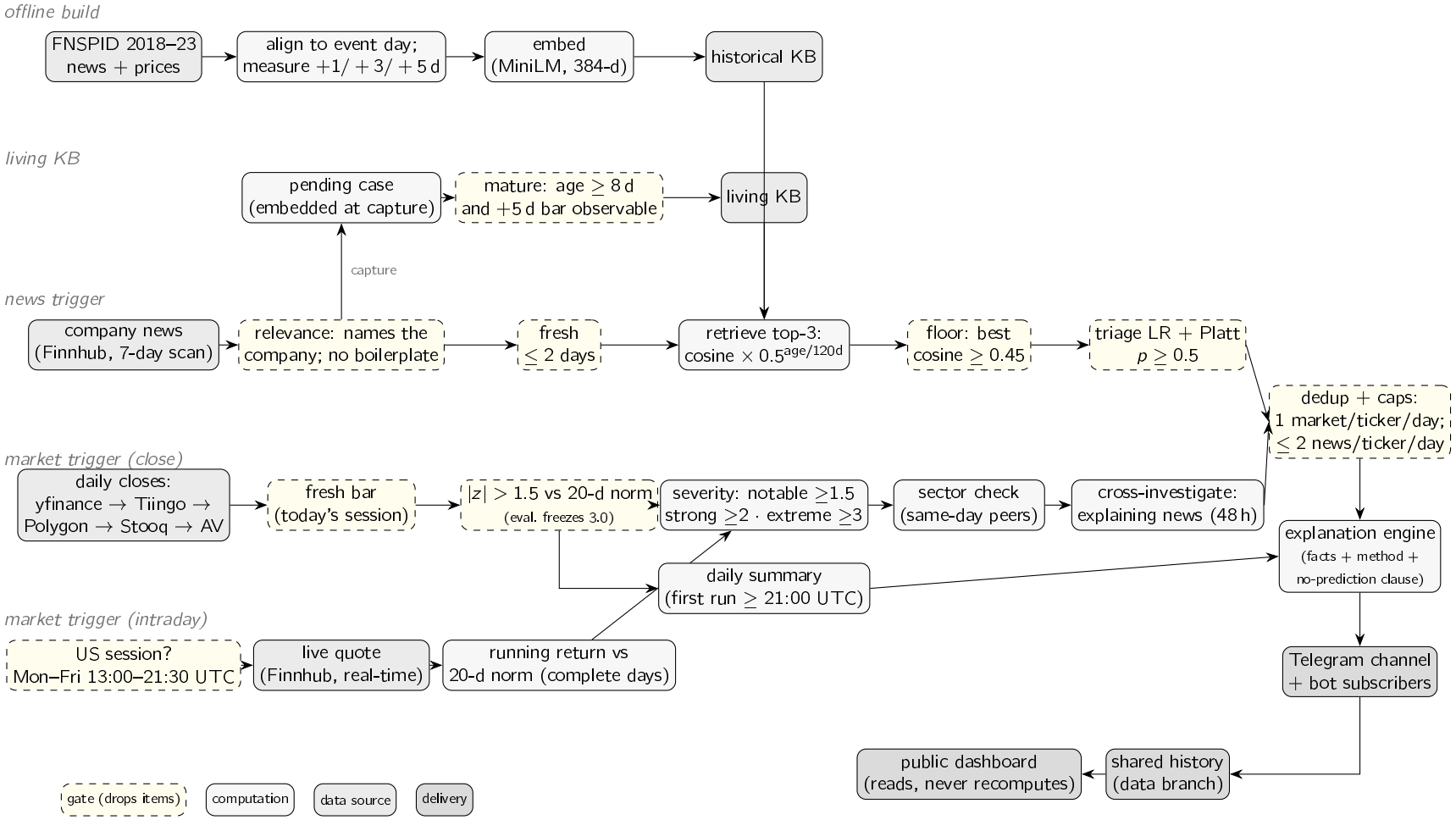

Figure A.1: The complete deployed pipeline on one page: the offline and living knowledge bases, the news trigger with its five quality gates, the close-based and intraday market triggers with the multi-source price fallback chain, and the shared outputs. Dashed boxes are gates that drop items; the annotated values (thresholds, caps, decay half-life) are the disclosed production configuration, distinct from the frozen evaluation settings of Chapter 5. The body chapters present the same system as several simpler views; this figure exists so that no stage is left implicit.

result, the value as reported, and the frozen file that a single command from Section A.4 regenerates. An examiner can reproduce any one number in isolation.

Table A.2: Each reported result, its value, and the frozen file one command regenerates.

Result	**Value**	**Frozen file**
Precedent retrieval, precision@5 (MiniLM / MPNet / lexical)	0.514 / 0.538 / 0.346	evaluation_results.md
Anomaly detection, F_1 (z-score vs fixed threshold)	0.516 vs 0.218	evaluation_anomaly.md
Statistical rule vs learned detectors, F_1 (z-score / IF / LOF)	0.530 / 0.269 / 0.280	evaluation_anomaly_ext.md
EWMA volatility (validated future), F_1 vs rolling	0.664 vs 0.516	evaluation_anomaly_ext.md
Materiality triage, PR-AUC (volatility / context / full)	0.542 / 0.538 / 0.496	evaluation_triage.md
Triage precision@5 per day vs alert-always	0.632 vs 0.163	evaluation_triage.md
Extended feature ablation, PR-AUC (context + 5 signals)	0.535 ($\Delta -0.002$)	evaluation_triage_ext.md
Worked triage decision (META, 12 Jul 2026)	$p = 0.539$ (54%)	triage_worked_example.md
Production selectivity (headlines : alerts sent)	22 : 1	alert_funnel.md

The system really ran. Beyond reproducibility on demand, InvestiGator operated live. A scheduled workflow scanned the watchlist after each US market close and delivered explained alerts to a public Telegram channel; the shared, versioned alert record accumulated dozens of real alerts, including the first genuine market anomaly (NVDA, 13 July 2026). The live knowledge base matured its captured headlines against the real prices that followed, and the post-validation loop labelled real past decisions with their observed outcome, reporting a kept-alert precision of 0.667 against a base rate of 0.455 — out-of-sample evidence that the triage mechanism holds. The automated suite of 199 tests, offline and deterministic, runs on every push through continuous integration, and the work is recorded across more than two hundred commits. Regenerable numbers, a live deployment, and a green test suite together are the dissertation's proof of work.

A.6 Data and Attribution

Large data files are never versioned; they are recreated by the data scripts, and only small samples are committed. The trained triage models are the exception by design: they are small, so the fitted bundles are versioned in the repository together with JSON metadata (training window, seed, threshold and metrics), which makes the deployed model itself inspectable and reproducible. The FNSPID dataset (Dong, Fan, and Peng 2024) is used under its CC BY-SA 4.0 licence, with attribution as required; the live market-data and news services are used within their free tiers.